

The University of Faisalabad

Data mining

Lab manual

Registration # 2023-BSAI-013 (6A)

Submitted to: Mr. Muhammad Arham

Submitted by: Seemal Mustafa

Department: Artificial intelligence

Table of Contents

LAB-1: Introduction to Data Mining Software & Datasets	5
Introduction to Data Mining Software	5
Overview of Datasets	5
LAB-2: CRISP-DM Process & Data Types	9
CRISP-DM Process	9
Types of Data	9
Real World Problem	10
Dataset	10
Practical Objectives	10
Procedure	10
Step 1: Business Understanding	10
Real-world Applications	13
LAB-3: Data Cleaning Techniques	14
1. Handling Missing Values	14
2. Removing Duplicates	15
3. Noise Handling using Python	16
LAB-4: Data Preprocessing	17
Normalization	17
Standardization	17
Discretization	17
Summary Statistics	17
RapidMiner Overview	20
LAB-5: Market Basket Analysis	24
Market Basket Analysis	24
1. Support	24
2. Confidence	24
3. Lift	24
LAB-6: Apriori Algorithm	27

Apriori Algorithm	27
Frequent Itemset Generation	27
Importing Libraries	27
LAB-7: FP-Tree & FP-Growth	34
FP-Tree Construction	34
FP-Growth Algorithm	34
LAB-8: Python Basics for Data Mining	41
Python Syntax	41
Example	42
OOP Basics (Object-Oriented Programming)	42
LAB 9: Decision Tree Classification	44
2. Dataset Overview	44
3. KPIs & Evaluation Metrics	44
KNIME Decision Tree Workflow	47
1. CSV Reader	47
2. Partitioning	48
3. Decision Tree Learner	48
4. Decision Tree Predictor	48
5. Scorer	48
Lab 10: Naïve Bayes & KNN – Email Spam Detection	50
Objective	50
Tool & Dataset	50
Workflow Steps	50
Step 1: Data Ingestion & Preparation (RapidMiner)	50
Step 2: Implementing Classifiers (RapidMiner)	51
Step 3: Prediction & Evaluation (RapidMiner + Python Integration)	51
Lab 11: SVM – Credit Card Fraud Detection	55
Objective	55
Tool & Dataset	55

Workflow Steps	55
Step 1: Data Ingestion & Preprocessing (KNIME)	55
Step 2: Training & Prediction (KNIME + Python Integration)	56
Lab 12: Clustering – Mall Customer Segmentation	59
Objective	59
To segment customers into distinct groups based on purchasing behavior using K-Means and Hierarchical clustering, and to evaluate these segments visually.	59
Tool & Dataset	59
Lab 13: Outlier Detection – Abnormal Banking Transactions	64
Workflow Steps	64
Step 1: Data Ingestion & Preprocessing (RapidMiner)	64
Step 2: Implementing Outlier Detection (RapidMiner)	64
Visualization & Analysis	65
Lab 14: Scraping & Sentiment Analysis – Product Review Analysis	68
Objective	68
Tool & Dataset	68
Workflow Steps	68
Lab 15: Final Capstone – End-to-End Business Solution	74
Objective	74
1. Data Preprocessing	74
2. Z- Score Normalization	75
3. Model Training	75
4. Modeling Pipeline	76
5. Model Evaluation	77
6. Outcome	77

LAB-1: Introduction to Data Mining Software & Datasets

Introduction to Data Mining Software

Data mining software refers to specialized applications and platforms that are used to extract useful patterns, relationships, and insights from large datasets. These tools are important because they convert raw and unprocessed data into meaningful information that can support better decision-making and future predictions.

Commonly used data mining tools include Python (along with libraries like Pandas, NumPy, and Scikit-learn), Knime, RapidMiner, Orange, and WEKA. These tools are popular in many industries because they can handle tasks such as data cleaning, visualization, classification, clustering, and discovering associations between data items.

For example, Python is widely preferred due to its flexibility and large collection of libraries, making it suitable for both beginners and experienced users. In contrast, RapidMiner provides an easy-to-use graphical interface, which is helpful for users who do not want to write much code.

Moreover, data mining software automates repetitive tasks and efficiently processes large datasets. In real-world scenarios, organizations use these tools to analyze customer behavior, predict future trends, and improve business strategies for better results.

Example:

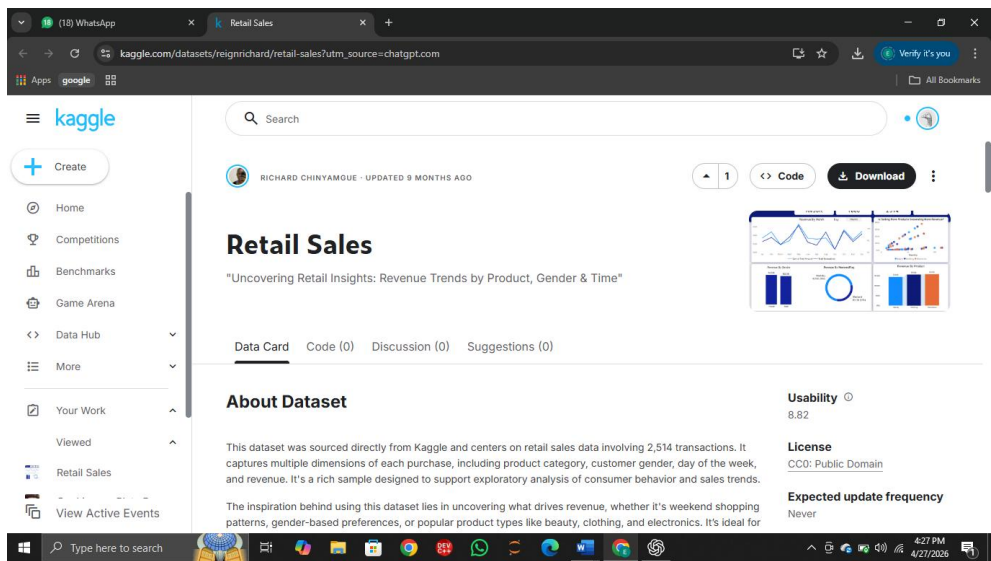
An e-commerce company can use data mining tools to examine customers' purchase history and suggest products based on their previous buying behavior.

Overview of Datasets

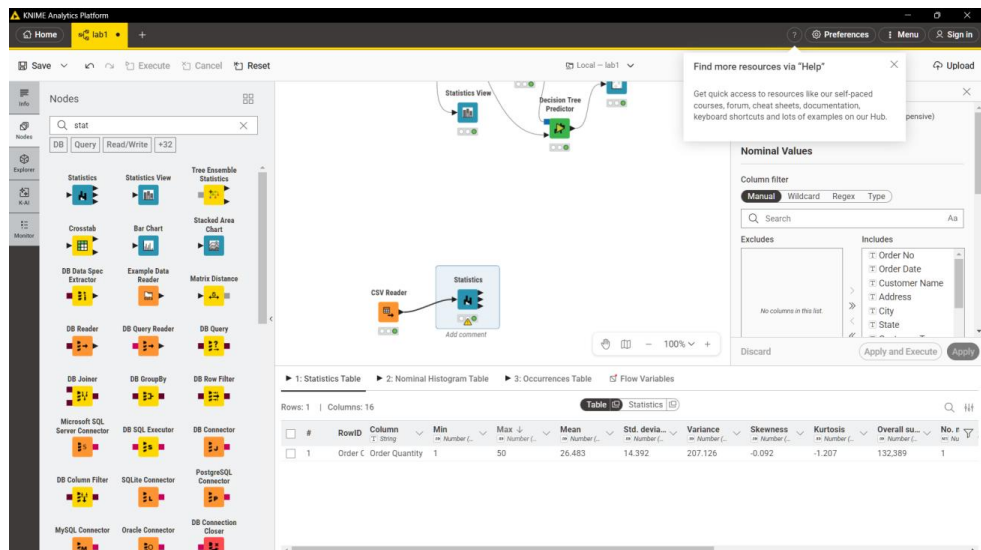
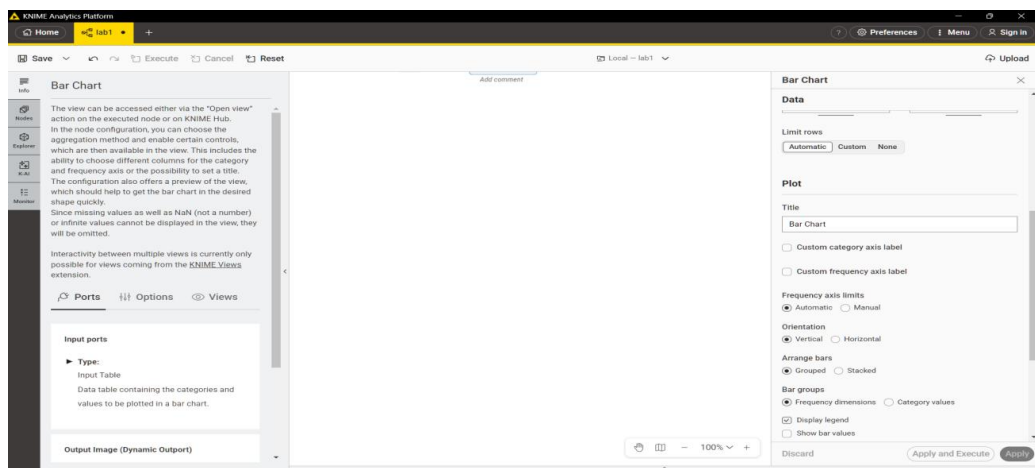
A dataset is a structured collection of data, usually organized in the form of rows and columns. In this arrangement, each row represents a single record, while each column corresponds to a specific attribute or feature.

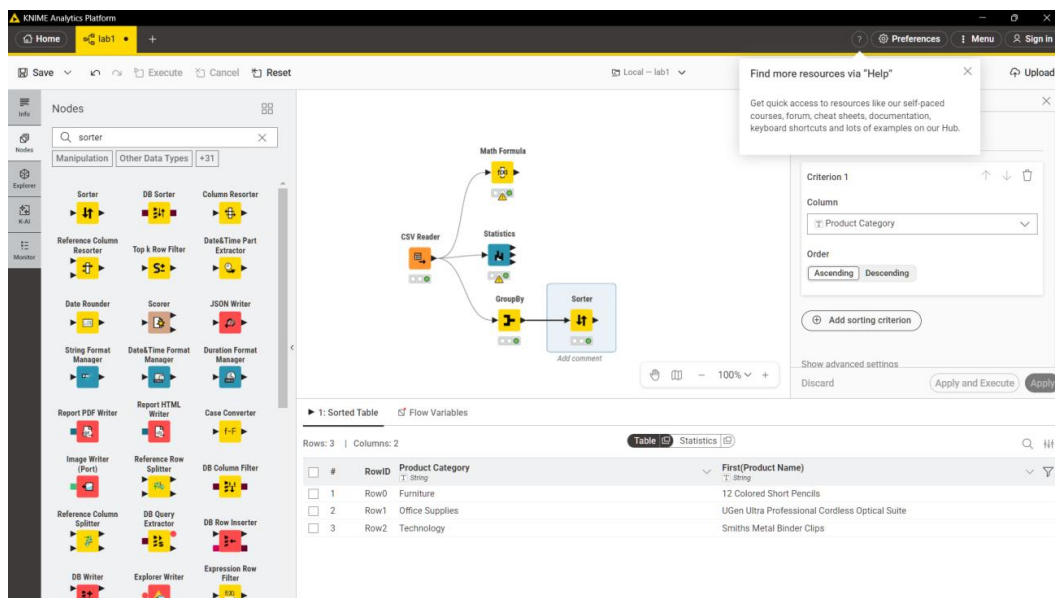
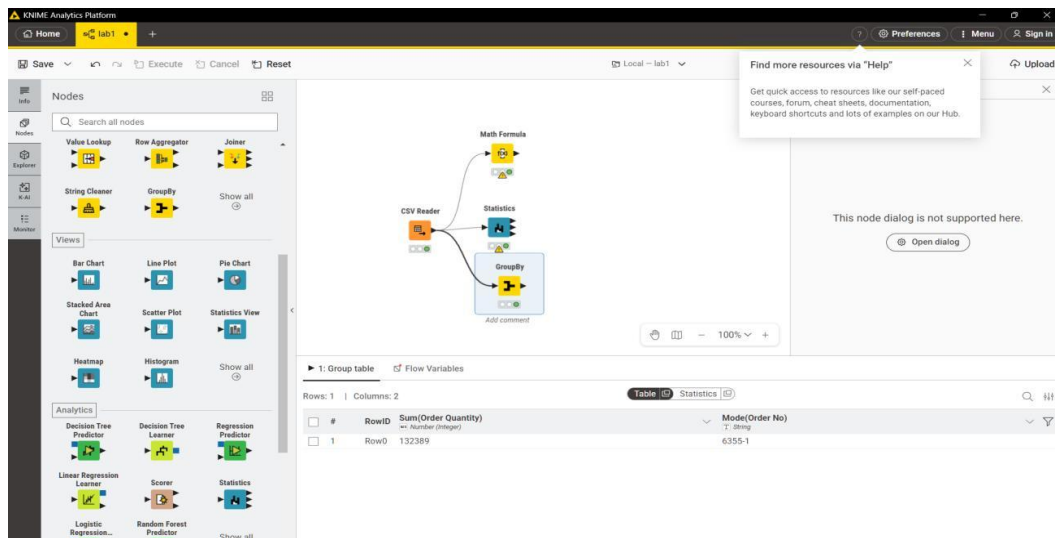
Datasets are generally divided into three main categories:

- **Structured Data:** Highly organized data typically stored in tables within databases.
- **Unstructured Data:** Data without a fixed format, such as text files, images, and videos.
- **Semi-structured Data:** Data that has some organizational structure but not as strict as structured data, commonly seen in formats like JSON and XML.



KNIME SOFTWARE :





Example:

A healthcare dataset might contain fields such as Patient ID, Age, Diagnosis, and Treatment Plan.

Understanding the type and structure of a dataset is important before applying data mining techniques. The quality of data directly affects the accuracy of the results, which is why preprocessing steps like data cleaning and transformation are necessary to improve model performance and ensure dependable outcomes.

LAB-2: CRISP-DM Process & Data Types

CRISP-DM Process

CRISP-DM is a step-by-step framework used in data mining projects:

The process is divided into six main phases:

1. **Business Understanding:**
This stage involves identifying the problem clearly, setting objectives, and understanding the overall requirements of the project.
2. **Data Understanding:**
In this phase, data is gathered and analyzed to explore patterns, gain insights, and identify issues such as missing or inconsistent values.
3. **Data Preparation:**
This step focuses on preparing the data by cleaning it, handling missing values, transforming variables, and organizing it for analysis.
4. **Modeling:**
At this stage, different machine learning or statistical techniques are applied to the prepared data to develop models for prediction or analysis.
5. **Evaluation:**
The created model is tested and evaluated to check whether it meets the defined goals and produces accurate results.
6. **Deployment:**
Finally, the model is implemented in a real-world environment where it can support decision-making or automate processes.

Example:

In a system designed to predict student performance, CRISP-DM provides a clear workflow—from identifying factors that influence grades to developing and deploying a model that forecasts student results.

Types of Data

Data can be classified into various types based on its nature and purpose:

- **Numerical Data:**
This type includes measurable values. It is further divided into:
 - Discrete (*e.g., number of subjects enrolled*)
 - Continuous (*e.g., temperature or height*)
- **Categorical Data:**
This represents descriptive or qualitative information and is divided into:

- Nominal (e.g., department names such as IT or HR)
 - Ordinal (e.g., levels like low, medium, high)
- **Binary Data:**
This type contains only two possible outcomes, such as yes/no or true/false.

Real World Problem

A subscription-based software company is experiencing an increase in customer cancellations. The management team wants to predict which customers are at risk of churning so that proactive retention strategies can be implemented. This lab applies the CRISP-DM (Cross-Industry Standard Process for Data Mining) framework to structure the entire data mining process from business understanding through deployment.

Dataset

Customer Subscription Dataset containing 50 records with customer demographics, subscription details, usage patterns, support interactions, and a binary churn label indicating whether the customer discontinued their subscription.

Practical Objectives

By the end of this lab, students will be able to:

0. Apply all six CRISP-DM phases to a real-world churn prediction problem.
1. Document business objectives and success criteria.
2. Explore and prepare datasets for modeling.
3. Build and evaluate predictive models using Orange Data Mining.
4. Develop a deployment plan for the solution.

Procedure

Step 1: Business Understanding

Define the business goal: predict customer churn. Identify success criteria: a model achieving at least 82% accuracy and an ROC-AUC score above 0.78. Document what churn means in this context (customer cancels subscription within 30 days) and list available data sources.

Open Orange Data Mining. Load the Customer Subscription CSV using the File widget. Connect a Data Table widget to inspect rows and columns. Use the Feature Statistics widget to view distributions, missing value counts, and class balance. Record the churn rate as a baseline metric.

Figure 2.1: Customer Churn Dataset Description

Customer churn refers to the phenomenon where customers discontinue their relationship or subscription with a company or service provider. It represents the rate at which customers stop using a company's products or services within a specific period. Churn is an important metric for businesses as it directly impacts revenue, growth, and customer retention.

In the context of the Churn dataset, the churn label indicates whether a customer has churned or not. A churned customer is one who has decided to discontinue their subscription or usage of the company's services. On the other hand, a non-churned customer is one who continues to remain engaged and retains their relationship with the company.

Understanding customer churn is crucial for businesses to identify patterns, factors, and indicators that contribute to customer attrition. By analyzing churn behavior and its associated features, companies can develop strategies to retain existing customers, improve customer satisfaction, and reduce customer turnover. Predictive modeling techniques can also be applied to forecast and proactively address potential churn, enabling companies to take proactive measures to retain at-risk customers.

customer_churn_dataset-testing-master.csv (3.28 MB)

Detail Compact Column

10 of 12 columns

About this file

Suggest Edits

The testing file for the churn dataset consists of 64,374 customer records and serves as a separate dataset for evaluating the performance and generalization capabilities of trained churn prediction models. Each record in the testing file corresponds to a customer and contains the same set of features as the training file, such as age, gender, tenure, usage frequency, support calls, payment delay, subscription type, contract length, total spend, and last interaction. However, the churn labels are not included in the testing file as they are used for assessing the accuracy and effectiveness of the churn prediction models. The testing file allows businesses to assess the predictive power of their trained models on unseen data and gain insights into how well the models generalize to new customers. By analyzing the model's performance on the testing file, businesses can gauge the effectiveness of their churn prediction strategies and make informed decisions to optimize customer retention efforts.

Use the Preprocess widget to handle missing values (impute with median for numeric, most frequent for categorical), encode categorical features using the Continuize widget, and normalize all numeric attributes. Remove any identifier columns such as Customer_ID that carry no predictive value.

Data Table - Orange

Info

64374 instances (no missing data)

12 features

No target variable.

No meta attributes.

Variables

☒ Show variable labels (if present)

☐ Visualize numeric values

☒ Color by instance classes

Selection

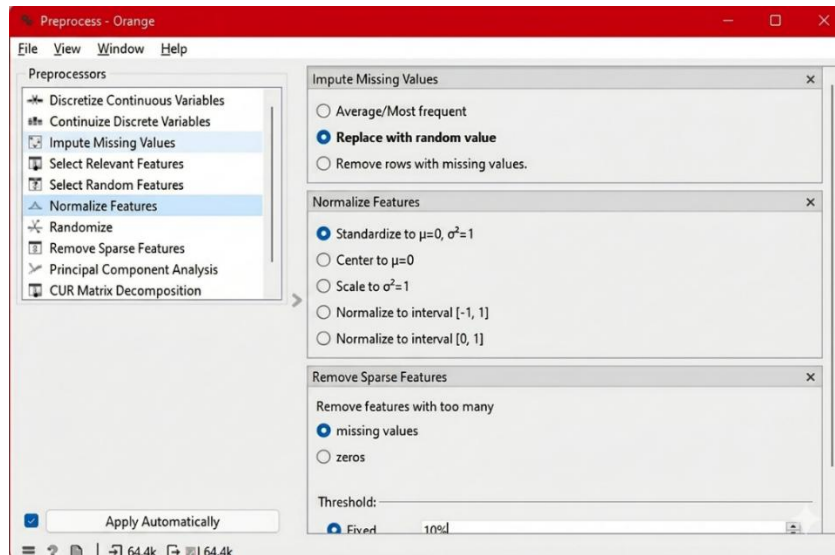
Clear Selection

☒ Select full rows

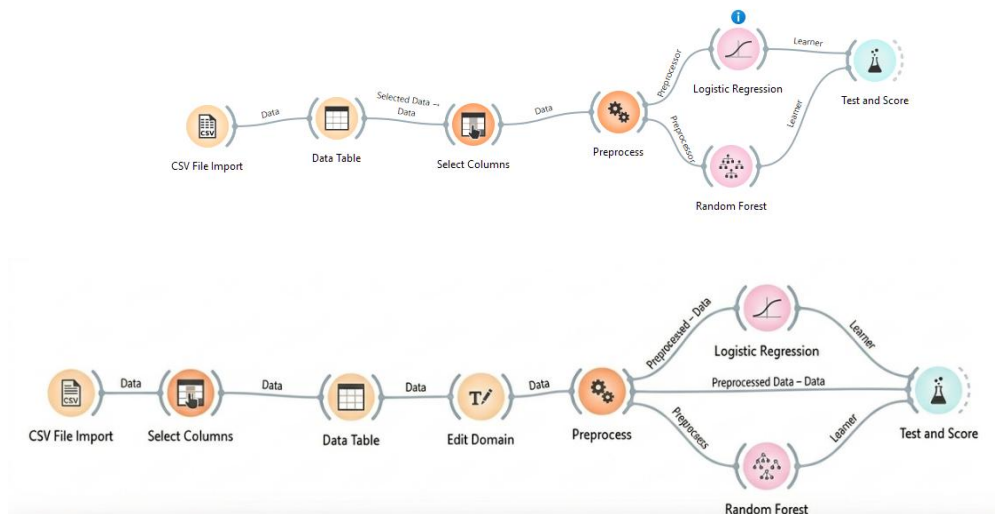
Restore Original Order

☒ Send Automatically

	CustomerID	Age	Gender	Tenure	Usage Frequency
1	1	30	Male	15	8
2	2	27	Male	3	2
3	3	47	Male	27	10
4	4	35	Male	9	12
5	5	29	Female	58	24
6	6	27	Female	41	14
7	7	47	Female	37	15
8	8	54	Female	36	11
9	9	36	Male	20	5
10	10	65	Male	8	4
11	11	46	Female	42	27
12	12	56	Male	13	23
13	13	31	Male	2	7
14	14	42	Male	46	27
16	15	59	Male	21	17
16	16	35	Female	1	3
17	17	29	Female	54	35
18	18	45	Male	9	30



Add a Logistic Regression learner and a Gradient Boosting learner to the canvas. Connect both to the preprocessed data. Link them to the Test and Score widget configured with 10-fold cross-validation and the Churned column set as the target variable.



Test and Score - Orange

Cross validation

- Number of folds: 10
- ☒ Stratified
- ☐ Cross validation by feature
- ☒ Stratified
- ☐ Random sampling
- Repeat train/test: 10
- Training set size: 66 %
- ☒ Stratified
- ☐ Leave one out

Evaluation results for target (None, show average over classes)

Model	AUC	CA	F1	Prec	Recall	MCC
Random Forest (Preproc. Gender - Random Value, Norm.)	0.998	0.985	0.985	0.986	0.985	0.969
Logistic Regression (Preproc. Gender - Random Value, N...)	0.992	0.970	0.970	0.971	0.970	0.941

Compare models by: Area under ROC curve

	Random For...	AUC	CA	F1	Prec	Recall	MCC
Random Forest	Logistic Reg...	0.998	0.985	0.985	0.986	0.985	0.969
Logistic Regression	Preproc. Data	0.992	0.970	0.970	0.971	0.970	0.941

Table shows probabilities that the score for the model in the row is higher than in the column. Small numbers show probability that the difference is negligible.

View the Test and Score results. Record Accuracy, Precision, Recall, F1, and ROC-AUC for both models. Connect the ROC Analysis widget to visualize the curves. Compare both models against the success criteria defined in Step 1 and select the better performer.

Evaluation results for target (None, show average over classes) ▾						
Model	AUC	CA	F1	Prec	Recall	MCC
Random Forest (Preproc. Gender - Random Value, Norm.)	0.998	0.985	0.985	0.986	0.985	0.969
Logistic Regression (Preproc. Gender - Random Value, N...	0.992	0.970	0.970	0.971	0.970	0.941

Real-world Applications

Data mining is widely used in various industries to extract valuable insights:

- **Healthcare:** Helps in disease prediction and supports medical diagnosis.
- **Retail:** Assists in recommending products and understanding customer purchasing behavior.
- **Finance:** Used for detecting fraud and managing financial risks.
- **Social Media:** Enables sentiment analysis and targeted advertising.

Example:

Streaming services like Netflix use data mining techniques to recommend movies and shows based on users' past viewing habits and preferences.

LAB-3: Data Cleaning Techniques

1. Handling Missing Values

Missing values occur when some data entries are absent or were not recorded. These missing pieces of information can affect data analysis and reduce the accuracy of models if they are not properly addressed.

There are several ways to handle missing data. One common method is to remove records that contain missing values. Another approach is to replace the missing values using statistical measures such as mean, median, or mode. More advanced techniques include interpolation or applying machine learning methods to estimate the missing data.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import warnings
warnings.filterwarnings('ignore')

df = pd.read_csv('healthcare_dataset.csv')

print('Dataset Shape:', df.shape)
print('\nFirst 5 rows:')
df.head()
```

Dataset Shape: (100, 7)

First 5 rows:

	PatientID	Age	Gender	BloodPressure	Glucose	BMI	Diagnosis
0	1	58	Female	116.785133	86.101378	20.288436	Healthy
1	2	71	Female	109.662087	96.806629	10.179522	Healthy
2	3	48	None	NaN	60.986073	32.617709	Healthy
3	4	34	Female	110.713960	150.008702	37.716026	Healthy
4	5	62	Female	125.342397	69.996707	25.287056	Healthy

Next steps: [Generate code with df_raw](#) [New interactive sheet](#)

```
Step 1: Load and Profile the Data

# Profile the dataset
print('--- DATASET PROFILE ---')
print(f'Total Rows : {df_raw.shape[0]}')
print(f'Total Columns : {df_raw.shape[1]}')
print(f'\nData Types:')
print(df_raw.dtypes)
print(f'\nMissing Values per Column:')
print(df_raw.isnull().sum())
print(f'\nTotal Missing Cells : {df_raw.isnull().sum().sum()}')
print(f'\nTotal Duplicate Rows : {df_raw.duplicated(subset="PatientID").sum()}')
```

--- DATASET PROFILE ---

Total Rows : 100
Total Columns : 7

Data Types:

PatientID	int64
Age	int64
Gender	object
BloodPressure	float64
Glucose	float64
BMI	float64
Diagnosis	object

dtype: object

Missing Values per Column:

PatientID	0
Age	0
Gender	9
BloodPressure	5
Glucose	18
BMI	8
Diagnosis	7

dtype: int64

Total Missing Cells : 39
Total Duplicate Rows: 18

Step 2: Handle Missing Values

```
df_clean = df_raw.copy()

# --- Numeric columns: fill with mean ---
numeric_cols = ['Glucose', 'BMI', 'BloodPressure']
for col in numeric_cols:
    before = df_clean[col].isnull().sum()
    df_clean[col].fillna(df_clean[col].mean(), inplace=True)
    after = df_clean[col].isnull().sum()
    print(f'{col}: {before} missing + filled with mean ({df_raw[col].mean():.2f}) + {after} missing')

# --- Categorical columns: fill with most frequent value ---
cat_cols = ['Gender', 'Diagnosis']
for col in cat_cols:
    before = df_clean[col].isnull().sum()
    most_freq = df_clean[col].mode()[0]
    df_clean[col].fillna(most_freq, inplace=True)
    after = df_clean[col].isnull().sum()
    print(f'{col}: {before} missing + filled with "{most_freq}" + {after} missing')

print(f'\nTotal missing after handling: {df_clean.isnull().sum().sum()}')

Glucose: 10 missing + filled with mean (101.08) + 0 missing
BMI: 8 missing + filled with mean (26.49) + 0 missing
BloodPressure: 5 missing + filled with mean (118.62) + 0 missing
Gender: 9 missing + filled with "Male" + 0 missing
Diagnosis: 7 missing + filled with "Healthy" + 0 missing
Total missing after handling: 0
```

Example:

If some records do not include monthly income, those missing values can be filled using the median income of the dataset to maintain consistency.

2. Removing Duplicates

Duplicate data appears when the same record is repeated multiple times in a dataset. This repetition can lead to incorrect analysis and biased results.

To maintain data quality, it is essential to identify and remove duplicate records during the preprocessing stage. This ensures the dataset remains accurate and reliable.

Step 3: Remove Duplicate Records

```
# Check duplicates BEFORE
dups_before = df_clean.duplicated(subset='PatientID').sum()
print(f'Duplicate rows BEFORE removal: {dups_before}')
print(f'Total rows BEFORE : {len(df_clean)}')

# Remove duplicates - keep first occurrence
df_clean = df_clean.drop_duplicates(subset='PatientID', keep='first')

dups_after = df_clean.duplicated(subset='PatientID').sum()
print(f'\nDuplicate rows AFTER removal : {dups_after}')
print(f'Total rows AFTER : {len(df_clean)}')
print(f'Rows removed : {dups_before}')

Duplicate rows BEFORE removal: 10
Total rows BEFORE : 100

Duplicate rows AFTER removal : 0
Total rows AFTER : 90
Rows removed : 10
```

Example:

If a sales transaction is recorded more than once, removing the duplicate entries will prevent errors in calculating total sales revenue.

3. Noise Handling using Python

Noise refers to unwanted or random variations in data that can hide useful patterns. It often shows up as outliers or irregular values.

Noise can be reduced using methods such as binning, regression, and filtering. Python libraries like Pandas and NumPy provide useful tools to detect and smooth out noisy data.

**Example:**

In a dataset with temperature readings, unusually high or low values can be removed or adjusted to improve the accuracy of analysis

LAB-4: Data Preprocessing

Normalization

Normalization is a method used to adjust data values so they fall within a fixed range, usually between 0 and 1. It is particularly helpful when different features in a dataset have different scales, ensuring that no single feature dominates the analysis.

Formula:

$$(X - \text{Min}) / (\text{Max} - \text{Min})$$

Example:

Variables such as product price and quantity sold can be normalized so that both have equal importance during analysis.

Standardization

Standardization is a technique that transforms data so that it has a mean of 0 and a standard deviation of 1. This approach is commonly used in machine learning when data is expected to follow a normal distribution.

Formula:

$$(X - \text{Mean}) / \text{Standard Deviation}$$

It is frequently applied in algorithms like Support Vector Machines (SVM) and K-Nearest Neighbors (KNN).

Example:

In a dataset of exam scores, standardization allows scores from different subjects to be compared fairly, even if they have different marking schemes.

Discretization

Discretization is the process of converting continuous data into a limited number of categories or intervals. This simplifies the data and can improve the performance of certain models.

Example:

Income values can be divided into categories such as low, medium, and high income.

Summary Statistics

Summary statistics provide a brief description of the main features of a dataset. These measures help in understanding the data distribution and identifying patterns or trends.

Common statistical measures include:

- **Mean:** The average value
- **Median:** The middle value
- **Mode:** The most frequently occurring value
- **Variance:** Measures how spread out the data is
- **Standard Deviation:** Indicates how much the values deviate from the mean

Example:

In a dataset of website traffic, summary statistics can help determine the average number of daily visitors and observe fluctuations over time.

1. Summary Statistics

```
[1]: import pandas as pd

# Load the dataset
df = pd.read_csv('loan_prediction_dataset.csv')
```

```
[9]: print("Summary Statistics:")
      display(df.describe())
```

Summary Statistics:

	Age	Income	Credit_Score	Loan_Amount	Loan_Term	Loan_Approved
count	2000.000000	2000.000000	2000.000000	2000.000000	2000.000000	2000.000000
mean	43.805500	84533.585000	577.055000	25460.315000	35.47800	0.171000
std	14.929203	37771.169751	157.525951	14116.737774	16.98587	0.376603
min	18.000000	20155.000000	300.000000	1060.000000	12.00000	0.000000
25%	31.000000	50925.250000	440.000000	13444.250000	24.00000	0.000000
50%	44.000000	84073.500000	578.500000	25446.000000	36.00000	0.000000
75%	56.000000	117523.250000	715.250000	37949.250000	48.00000	0.000000
max	69.000000	149992.000000	849.000000	49994.000000	60.00000	1.000000

2. Normalization (Min-Max Scaling)

```
[4]: from sklearn.preprocessing import MinMaxScaler

# Initialize the scaler
scaler_minmax = MinMaxScaler()

# We will normalize 'Income' and 'Loan_Amount'
df_normalized = df.copy()
cols_to_normalize = ['Income', 'Loan_Amount']
df_normalized[cols_to_normalize] = scaler_minmax.fit_transform(df[cols_to_normalize])

print("Normalized Data (First 5 rows):")
print(df_normalized[cols_to_normalize].head())
```

Normalized Data (First 5 rows):

	Income	Loan_Amount
0	0.474695	0.285323
1	0.637137	0.407753
2	0.297850	0.789369
3	0.824364	0.323415
4	0.042361	0.514898

3. Standardization (Z-score Scaling)

```
[5]: from sklearn.preprocessing import StandardScaler

# Initialize the scaler
scaler_std = StandardScaler()

# We will standardize 'Age' and 'Credit_Score'
df_standardized = df.copy()
cols_to_standardize = ['Age', 'Credit_Score']
df_standardized[cols_to_standardize] = scaler_std.fit_transform(df[cols_to_standardize])

print("Standardized Data (First 5 rows):")
print(df_standardized[cols_to_standardize].head())
```

Standardized Data (First 5 rows):

	Age	Credit_Score
0	0.817026	-1.543338
1	1.688020	1.294999
2	0.147031	1.282300
3	-0.790963	-1.352846
4	1.085024	-1.714781

4. Discretization (Binning)

```
[7]: df_discretized = df.copy()
df_discretized['Income_Category'] = pd.cut(df['Income'],
                                           bins=3,
                                           labels=['Low', 'Medium', 'High'])

from sklearn.preprocessing import KBinsDiscretizer
est = KBinsDiscretizer(n_bins=3, encode='ordinal', strategy='uniform')
df_discretized['Age_Bins'] = est.fit_transform(df[['Age']])

print("Discretized Data (First 5 rows):")
print(df_discretized[['Income', 'Income_Category', 'Age', 'Age_Bins']].head())
```

```
Discretized Data (First 5 rows):
   Income Income_Category  Age  Age_Bins
0   81788             Medium   56      2.0
1  102879             Medium   69      2.0
2   58827              Low    46      1.0
3  127188              High   32      0.0
4   25655              Low    60      2.0
```

RAPID MINER :

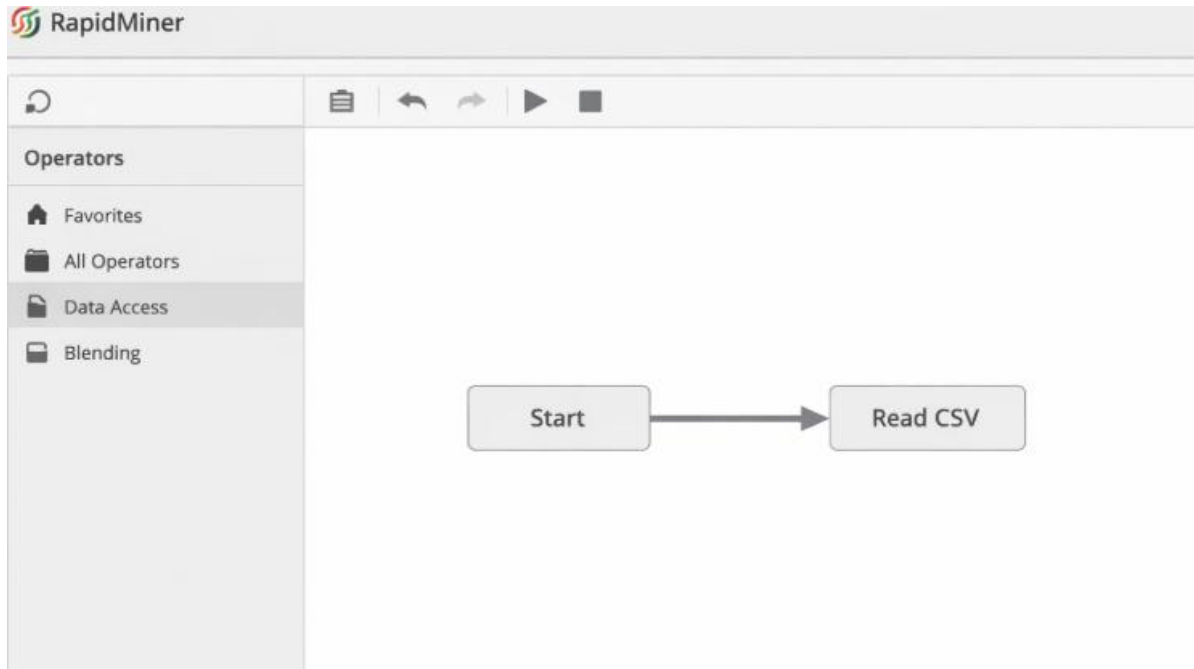
RapidMiner is a powerful, user- friendly data science platform designed for data preparation, machine learning, and predictive analytics. It uses a visual workflow approach where tasks are represented as “operators” connected in a process, making it ideal for lab manuals and teaching.

RapidMiner Overview

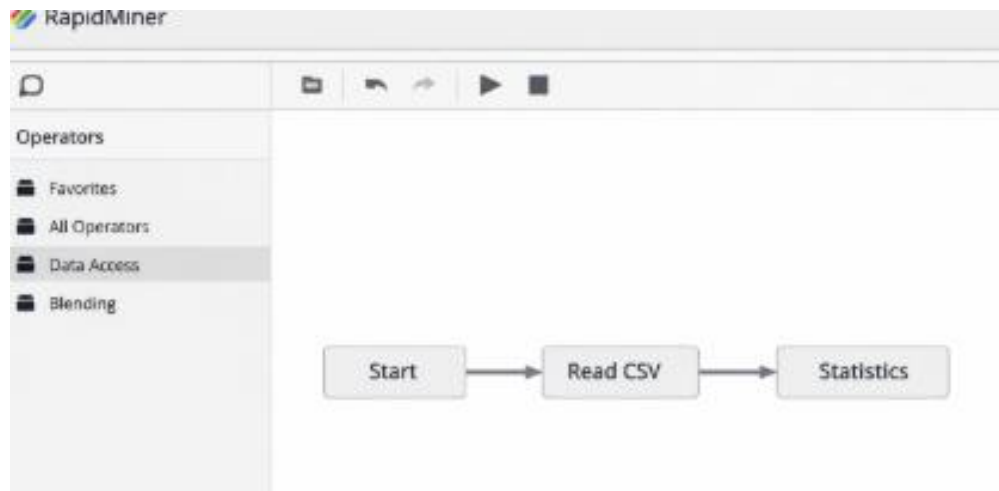
- Type: Data science and machine learning platform.
- Developed: Originally in 2001 at TU Dortmund; commercialized in 2006.
- Current Owner: Acquired by Altair Engineering in 2022, later by Siemens in 2025.
- Interface: Graphical workflow builder (“Processes”) with drag- and- drop operators.
- Programming Language: Written in Java, but extensible with R and Python scripts.
- Deployment: Available on- premises or in cloud environments.

RAPID MINER WORKFLOW:

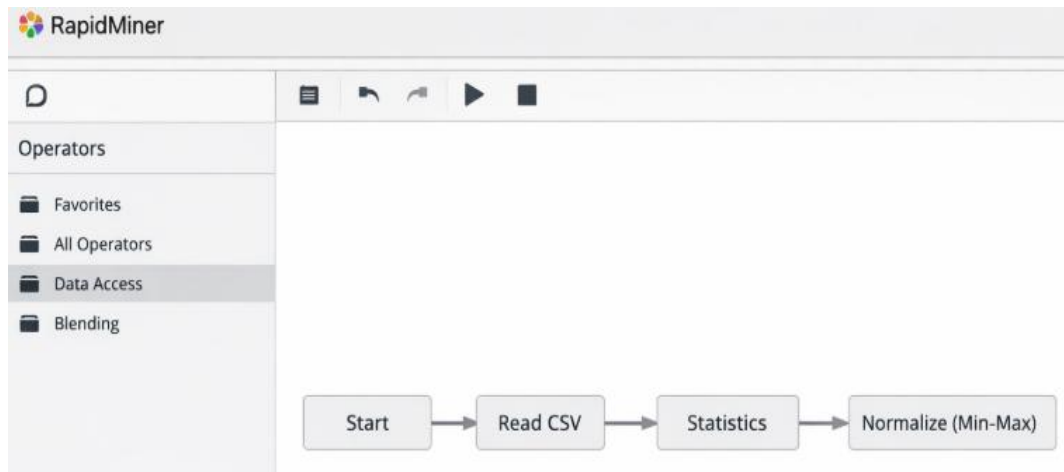
STEP 1: Start → Read CSV connected by an arrow.



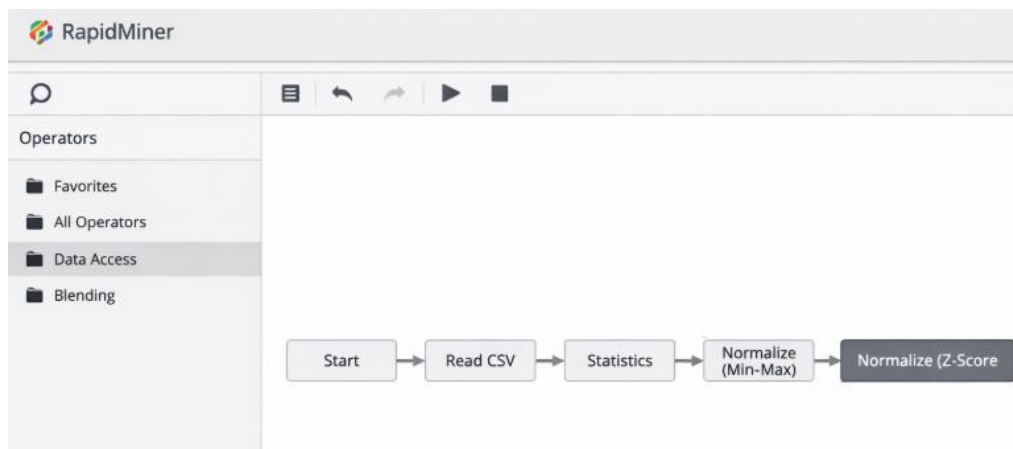
STEP 2: Start → Read CSV → Statistics.



STEP 3: Start → Read CSV → Statistics → Normalize (Min- Max).



STEP 4: Start → Read CSV → Statistics → Normalize (Min- Max) → Normalize (Z- Score).



STEP5:
Start → Read CSV → Statistics → Normalize (Min- Max) → Normalize (Z- Score) → Discretize.



LAB-5: Market Basket Analysis

Market Basket Analysis

Market Basket Analysis is a data mining method used to identify relationships between items that are often bought together. It helps businesses understand customer buying behavior and supports better decisions related to product placement, marketing strategies, and recommendations.

Example:

In a supermarket dataset, customers who purchase tea may also frequently buy biscuits, showing a strong connection between these two items.

1. Support

Support measures how frequently a particular item or group of items appears in the dataset. It indicates the popularity of an itemset.

Formula:

$\text{Support (X)} = \text{Transactions containing X} / \text{Total Transactions}$

Example:

If milk appears in 20 out of 100 transactions, then:

$\text{Support (milk)} = 20 / 100 = 0.2$

2. Confidence

Confidence represents the likelihood that item Y is purchased when item X has already been purchased. It reflects the strength of the relationship between two items.

Formula:

$\text{Confidence (X} \rightarrow \text{Y)} = \text{Support (X} \cup \text{Y)} / \text{Support (X)}$

Example:

If 15 transactions include both tea and biscuits, and 20 include tea, then:

$\text{Confidence (tea} \rightarrow \text{biscuits)} = 15 / 20 = 0.75$

3. Lift

Lift measures how strong the association between two items is compared to when they are independent. It helps determine whether the relationship is meaningful.

Formula:

$\text{Lift}(X \rightarrow Y) = \text{Confidence}(X \rightarrow Y) / \text{Support}(Y)$

- **Lift > 1:** Positive relationship (items are related)
- **Lift < 1:** Negative relationship (items rarely occur together)
- **Lift = 1:** No relationship (items are independent)

Example:

If Confidence (tea $\rightarrow$ biscuits) = 0.75 and Support (biscuits) = 0.5, then:

Lift = $0.75 / 0.5 = 1.5 \rightarrow$ indicating a strong positive association

Step 1: Load Data

```
[1]: import pandas as pd

df = pd.read_csv("transaction_data.csv")
df.head()
```

	UserId	TransactionId	TransactionTime	ItemCode	ItemDescription	NumberOfItemsPurchased	CostPerItem	Country
0	278166	6355745	Sat Feb 02 12:50:00 IST 2019	465549	FAMILY ALBUM WHITE PICTURE FRAME	6	11.73	United Kingdom
1	337701	6283376	Wed Dec 26 09:06:00 IST 2018	482370	LONDON BUS COFFEE MUG	3	3.52	United Kingdom
2	267099	6385599	Fri Feb 15 09:45:00 IST 2019	490728	SET 12 COLOUR PENCILS DOLLY GIRL	72	0.90	France
3	380478	6044973	Fri Jun 22 07:14:00 IST 2018	459186	UNION JACK FLAG LUGGAGE TAG	3	1.73	United Kingdom
4	-1	6143225	Mon Sep 10 11:58:00 IST 2018	1733592	WASHROOM METAL SIGN	3	3.40	United Kingdom

Step 2: Convert Data into Transaction List

```
[5]: transactions = df.groupby('TransactionId')['ItemDescription'].apply(list)
transactions = list(transactions)

print(transactions[:5])
```

```
[[ 'SET 7 BABUSHKA NESTING BOXES', 'WHITE METAL LANTERN', 'RED WOOLLY HOTTIE WHITE HEART.', 'CREAM CUPID HEARTS COAT HANGER', 'SET 7 BABUSHKA NESTING BOXES', 'KNITTED UNION FLAG HOT WATER BOTTLE', 'KNITTED UNION FLAG HOT WATER BOTTLE', 'WHITE HANGING HEART T-LIGHT HOLDER', 'WHITE METAL LANTERN', 'CREAM CUPID HEARTS COAT HANGER', 'RED WOOLLY HOTTIE WHITE HEART.', 'GLASS STAR FROSTED T-LIGHT HOLDER', 'WHITE HANGING HEART T-LIGHT HOLDER', 'GLASS STAR FROSTED T-LIGHT HOLDER'], ['HAND WARMER RED POLKA DOT', 'HAND WARMER UNION JACK', 'HAND WARMER UNION JACK', 'HAND WARMER RED POLKA DOT'], ['DOORMAT NEW ENGLAND', 'HOME BUILDING BLOCK WORD', 'LOVE BUILDING BLOCK WORD', 'ASSORTED COLOUR BIRD ORNAMENT', 'FELTCRAFT PRINCESS CHARLOTTE DOLL', 'POPPY'S PLAYHOUSE BEDROOM', 'LOVE BUILDING BLOCK WORD', 'ASSORTED COLOUR BIRD ORNAMENT', 'BOX OF 6 ASSORTED COLOUR TEASPOONS', 'RECIPE BOX WITH METAL HEART', 'IVORY KNITTED MUG COSY', 'BOX OF VINTAGE JIGSAW BLOCKS', 'BOX OF VINTAGE ALPHABET BLOCKS', 'POPPY'S PLAYHOUSE BEDROOM', 'BOX OF VINTAGE ALPHABET BLOCKS', 'POPPY'S PLAYHOUSE KITCHEN', 'FELTCRAFT PRINCESS CHARLOTTE DOLL', 'DOORMAT NEW ENGLAND', 'HOME BUILDING BLOCK WORD', 'IVORY KNITTED MUG COSY', 'RECIPE BOX WITH METAL HEART', 'BOX OF 6 ASSORTED COLOUR TEASPOONS', 'POPPY'S PLAYHOUSE KITCHEN', 'BOX OF VINTAGE JIGSAW BLOCKS'], ['BLUE COAT RACK PARIS FASHION', 'RED COAT RACK PARIS FASHION', 'YELLOW COAT RACK PARIS FASHION', 'JAM MAKING SET WITH JARS', 'YELLOW COAT RACK PARIS FASHION', 'RED COAT RACK PARIS FASHION', 'BLUE COAT RACK PARIS FASHION', 'JAM MAKING SET WITH JARS'], ['BATH BUILDING BLOCK WORD', 'BATH BUILDING BLOCK WORD']]
```

```
[6]: transactions = [list(set(t)) for t in transactions]
```

Step 3: Total Transactions

```
[7]: total_transactions = len(transactions)
    print(total_transactions)
```

25900

Step 4: Manual Calculation ($A \rightarrow B$)

```
[8]: A = 'WHITE HANGING HEART T-LIGHT HOLDER'
    B = 'CREAM CUPID HEARTS COAT HANGER'
```

```
[9]: count_A = 0
    count_B = 0
    count_AB = 0

    for t in transactions:
        if A in t:
            count_A += 1
        if B in t:
            count_B += 1
        if A in t and B in t:
            count_AB += 1

    support = count_AB / total_transactions
    confidence = count_AB / count_A if count_A != 0 else 0
    lift = confidence / (count_B / total_transactions) if count_B != 0 else 0

    print("Support:", support)
    print("Confidence:", confidence)
    print("Lift:", lift)
```

Support: 0.004362934362934363
Confidence: 0.04908774978279757
Lift: 4.384043859911921

LAB-6: Apriori Algorithm

Apriori Algorithm

The Apriori algorithm is a commonly used technique in data mining for discovering frequent itemsets in transactional datasets. It is based on an important concept: if an itemset is considered frequent, then all of its smaller subsets must also be frequent.

The algorithm follows an iterative process:

- It begins by identifying single items that satisfy the minimum support threshold
- Next, it creates larger combinations of items (called candidate itemsets)
- Itemsets that do not meet the minimum support are eliminated (pruned)

This process is repeated until no additional frequent itemsets can be generated.

Example:

In a grocery dataset, if the combination {Milk, Eggs} is frequently purchased together, then {Milk} and {Eggs} must also appear frequently on their own.

Frequent Itemset Generation

Frequent itemsets are groups of items that occur together often in a dataset, based on a specified support threshold. These itemsets are important because they are used to generate association rules.

Example:

In an online shopping dataset, {Laptop, Mouse} can be considered a frequent itemset if customers regularly buy these items together.

Importing Libraries

Before implementing the Apriori algorithm in Python, it is necessary to import the required libraries that assist in data handling and analysis.

```
[2]: import pandas as pd
      from mlxtend.preprocessing import TransactionEncoder
      from mlxtend.frequent_patterns import apriori, association_rules
```

Load Your Dataset

```
[3]: df = pd.read_csv('transaction_data.csv')
df.head()
```

	Userld	Transactionld	TransactionTime	ItemCode	ItemDescription	NumberOfItemsPurchased	CostPerItem	Country
0	278166	6355745	Sat Feb 02 12:50:00 IST 2019	465549	FAMILY ALBUM WHITE PICTURE FRAME	6	11.73	United Kingdom
1	337701	6283376	Wed Dec 26 09:06:00 IST 2018	482370	LONDON BUS COFFEE MUG	3	3.52	United Kingdom
2	267099	6385599	Fri Feb 15 09:45:00 IST 2019	490728	SET 12 COLOUR PENCILS DOLLY GIRL	72	0.90	France
3	380478	6044973	Fri Jun 22 07:14:00 IST 2018	459186	UNION JACK FLAG LUGGAGE TAG	3	1.73	United Kingdom
4	-1	6143225	Mon Sep 10 11:58:00 IST 2018	1733592	WASHROOM METAL SIGN	3	3.40	United Kingdom

```
[6]: df = df.dropna()

# Remove negative or zero quantities (if exist)
df = df[df['NumberOfItemsPurchased'] > 0]

df.head()
```

	Userld	Transactionld	TransactionTime	ItemCode	ItemDescription	NumberOfItemsPurchased	CostPerItem	Country
0	278166	6355745	Sat Feb 02 12:50:00 IST 2019	465549	FAMILY ALBUM WHITE PICTURE FRAME	6	11.73	United Kingdom
1	337701	6283376	Wed Dec 26 09:06:00 IST 2018	482370	LONDON BUS COFFEE MUG	3	3.52	United Kingdom
2	267099	6385599	Fri Feb 15 09:45:00 IST 2019	490728	SET 12 COLOUR PENCILS DOLLY GIRL	72	0.90	France
3	380478	6044973	Fri Jun 22 07:14:00 IST 2018	459186	UNION JACK FLAG LUGGAGE TAG	3	1.73	United Kingdom
4	-1	6143225	Mon Sep 10 11:58:00 IST 2018	1733592	WASHROOM METAL SIGN	3	3.40	United Kingdom

Convert into Basket Format

```
[7]: basket = df.groupby(['TransactionId', 'ItemDescription'])['NumberOfItemsPurchased'] \
        .sum().unstack().reset_index().fillna(0)

basket.head()
```

	ItemDescription	TransactionId	4 PURPLE FLOCK DINNER CANDLES	50'S CHRISTMAS GIFT BAG LARGE	DOLLY GIRL BEAKER	I LOVE LONDON MINI BACKPACK	I LOVE LONDON MINI RUCKSACK	NINE DRAWER OFFICE TIDY	OVAL WALL MIRROR DIAMANTE	RED SPOT GIFT BAG LARGE	SET 2 TEA TOWELS I LOVE LONDON	...	returned	taig adjust	test
0		5900015	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0
1		5900026	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0
2		5900037	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0
3		5900048	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0
4		5900059	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0

5 rows × 4078 columns

Convert Values into 0/1 (Binary Encoding)

```
[8]: def encode(x):
      if x > 0:
          return 1
      else:
          return 0

      basket_encoded = basket.set_index('TransactionId').applymap(encode)

      basket_encoded.head()
```

C:\Users\star\AppData\Local\Temp\ipykernel_11340\1282610810.py:7: FutureWarning: DataFrame.applymap has been deprecated. Use DataFrame.map instead

```
      basket_encoded = basket.set_index('TransactionId').applymap(encode)
```

```
[8]:
```

ItemDescription	4 PURPLE FLOCK DINNER CANDLES	50'S CHRISTMAS GIFT BAG LARGE	DOLLY GIRL BEAKER	I LOVE LONDON MINI BACKPACK	I LOVE LONDON MINI RUCKSACK	NINE DRAWER OFFICE TIDY	OVAL WALL MIRROR DIAMANTE	RED SPOT GIFT BAG LARGE	SET 2 TEA TOWELS I LOVE LONDON	SPACEBOY BABY GIFT SET	...	returned	taig adjust	test
TransactionId														
5900015	0	0	0	0	0	0	0	0	0	0	...	0	0	0
5900026	0	0	0	0	0	0	0	0	0	0	...	0	0	0

Apply Apriori Algorithm

```
[10]: rules = association_rules(frequent_itemsets,
                                metric="confidence",
                                min_threshold=0.6)

      rules.head()
```

```
[10]:
```

antecedents	consequents	antecedent support	consequent support	support	confidence	lift	representativity	leverage	conviction	zhangs_metric	jaccard	certainty	kulczynski
-------------	-------------	-----------------------	-----------------------	---------	------------	------	------------------	----------	------------	---------------	---------	-----------	------------

Display Important Rules

```
[11]: rules = rules.sort_values(by='confidence', ascending=False)

      rules[['antecedents', 'consequents', 'support', 'confidence', 'lift']]
```

```
[11]:
```

antecedents	consequents	support	confidence	lift
-------------	-------------	---------	------------	------

Filter Strong Rules

```
[12]: strong_rules = rules[(rules['confidence'] > 0.7) & (rules['lift'] > 1)]

      strong_rules
```

```
[12]:
```

antecedents	consequents	antecedent support	consequent support	support	confidence	lift	representativity	leverage	conviction	zhangs_metric	jaccard	certainty	kulczynski
-------------	-------------	-----------------------	-----------------------	---------	------------	------	------------------	----------	------------	---------------	---------	-----------	------------

KNIME

KNIME (pronounced “*naim*”), which stands for Konstanz Information Miner, is a free, open-source data analytics, reporting, and integration platform. It operates as a low-code/no-code environment, allowing data professionals, engineers, and domain experts to access, blend, clean, and analyze complex datasets using a highly visual environment without necessarily writing code.

KNIME WORKFLOW:

STEP 1: CSV Reader – Loading the Transaction Dataset

The screenshot displays the KNIME workflow interface. A 'CSV Reader' node is connected to a 'Dataset' node. The 'CSV Reader - Configuration' dialog box is open, showing the 'File path' as 'C:\Users\user\data\transaction_data.csv'. The 'Preview Tables' section shows a table with columns: File, Stériden, ItemDescription, NumbersOtrim..., and NumPertiam... The 'Data Output' table at the bottom shows the loaded data with columns: Userid, TransactionId, ItemsDescription, ItemDescription, Number..., NomPertiam, and Oturtivu.

File	Stériden	ItemDescription	NumbersOtrim...	NumPertiam...
Item 1	2022-05-23 136...	oine	38.00	
Item 2	2025-06-23 136...	Kem6description	28.00	
Item 3	2020-06-23 125...	Item	68.00	
Item 4	2020-06-23 155...	Scend	48.00	
Item 5	2020-06-23 155...	Seed	58.00	

Userid	TransactionId	ItemsDescription	ItemDescription	Number...	NomPertiam	Oturtivu
Item 1	278166	202-05-25 13 13:35:36...	FAMILY ALBUM WHITE PICTURE FRA...	5.00	5.50 Together	
Item 2	278166	202-05-23 13 13:33:32...	GUPVam C&G GATTER FALE	5.00	3.50 Together	
Item 2	278166	202-05-23 13 13:33:36...	FAMILY ALBUM WHITE PICTUET RGR...	5.00	Indens	
Item 3	278126	202-05-23 13 15:35:36...	FAMILY ALBUM WRESGE KREUT ABIL	5.00	Indens	
Item 4	278130	202-05-23 13 13:33:36...	ROMSMON PASIVATRE	5.00	Indens	
Item 5	278432	202-05-23 13 13:33:36...	ROMSMON JAAG LAG TABLE	5.00	Indens	
Item 6	278876	2020-5-23 13 13:39:36...	Nones	5.00	Indens	
Item 7	278590	2020-5-23 13 13:39:53...	atsar	5.00	Indens	
Item 8	278576	2020-5-23 13 13:39:35...	ulido	5.00	Indens	
Item 9	200598	2020-5-23 13 13:33:49...	ulido	5.00	Indens	
Item 9	200590	202-05-23 13 13:39:36...	Nexnescriptazed Toiletad:	5.00	Indens	
Item 0	200510	202-05-23 13 13:45:46...	FAMILY ALBUM MEIT FAGFTIARE	5.00	3.50 Together	

STEP 2: Missing Value & Row Filter – Handling Nulls and Negative Quantities

Missing Value - Configuration

Configuration | Katos | Liberanics | Data

Columnant: Configuration

Configuration: Remove Row

Missing Data: Kamara Row

M Description:

☐ Remove e nissing Data

☐ Remove enstoves to missine Kloe

OK Cancel

Data Output

UserId	TransactionId	ItemDescription
Row 1	Transactionlon	0
Row 2	Gersorsanddere	0
Rpw 3	Gersorsandosse	5
Rpw 4	sbxar	0
Rpw 4	uldb	0

STEP 3: Column Filter

Column Filter - Configuration

Configen | Headen | Sastar | Data | Detention

Select the columns to into selectsted columns.

All Columns:

- All Columns
- ItemCode
- ItemDescription
- ItemDescription
- ItemPerfiam
- ItemCode
- Item Taka
- NumberOfItemCalust
- NumberOfItemsPurchased
- BasePerfTirm

Selected Columns:

- TransactionTime
- ItemCode
- ItemDescription
- NumberOfItemsPurchased

OK Cancel

Data Output

UserId	TransactionId	ItemDescription	ItemDescription	NumberOfItemsPurchased
Item 1	278166	202-06-23 13 13:33:46...	TransactionTime	380
Item 2	278166	202-06-23 13 13:33:36...	FAMILY ALBUM WHITE PICTUR ...	300
Item 2	337701	Wed 06-28 15 13:33:46...	ItemCode	100
Item 3	278795	202-06-23 13 15:33:36...	LONDON BUS COFFEE MUG	5
Item 4	628989	Fr/4 06-22 15 13:33:56...	ST LOU COLOUR DOLLY DOLLY ...	72
Item 5	278129	202-06-23 13 13:33:36...	ROMISON JACK WRESSE KREUT...	5
Item 6	614325	Ron 06-21 11 15:33:46...	WASHRON JACK LAG TWSIS	3

STEP 4: Create Collection Column

KNIME

Create Collection Column - Configuration

Configuration | Katos | Libraries | Data

Group by: TransactionId

Column to collect: ItemDescription

Preview Tables

File	Item List
Row 1 (278166)	(FAMILY ALBUM WHITE PICTURE FRAME, ... LONDON BUS COFFEE MUG), LIVEIZERI...
Row 2 (337701)	(LONDON BUS COFFEE MUG)
Row 3 (271626)	ROMSMON FASIVATVER
Row 4 (278436)	ROMSMON JAAG LAG TABLE
Row 5 (278850)	star
Row 6 (200599)	ROMSION JACK LAG TAG2 LAGG

OK Cancel ?

Data Output

ID	TransactionId	Collected Items
Row 1	278166	[FAMILY ALBUM WHITE PICTURE FRAME, ... LONDON BUS COFFEE MUG], LIVEIZERI...
Row 2	337701	[LONDON BUS COFFEE MUG]
Row 3	278509	[FAMILY ALBUM WHITE PICTURE FRAME, ... LONDON BUS COFFEE MUGS] 490776R
Row 4	278126	[FAMILY ALBUM WRESSE KREUT ARIL, ... [AASHRON BUS COFFEE BAG, RED, SET T...
Row 4	278432	[ROMSMON FASIVATVAR LADNS, ROMSKONN WHITE KREUT BAG, RED SET TKA...
Row 5	200599	[WASHRON JACK LAG TAGSE RXL DROEL LOCK MINI BAG, UNMEFDOT LAG LARGE...

STEP 5: Pivoting & One-Hot Encoding

Pivoting - Configuration

Configuration | Header | Sastar | Data | Detection

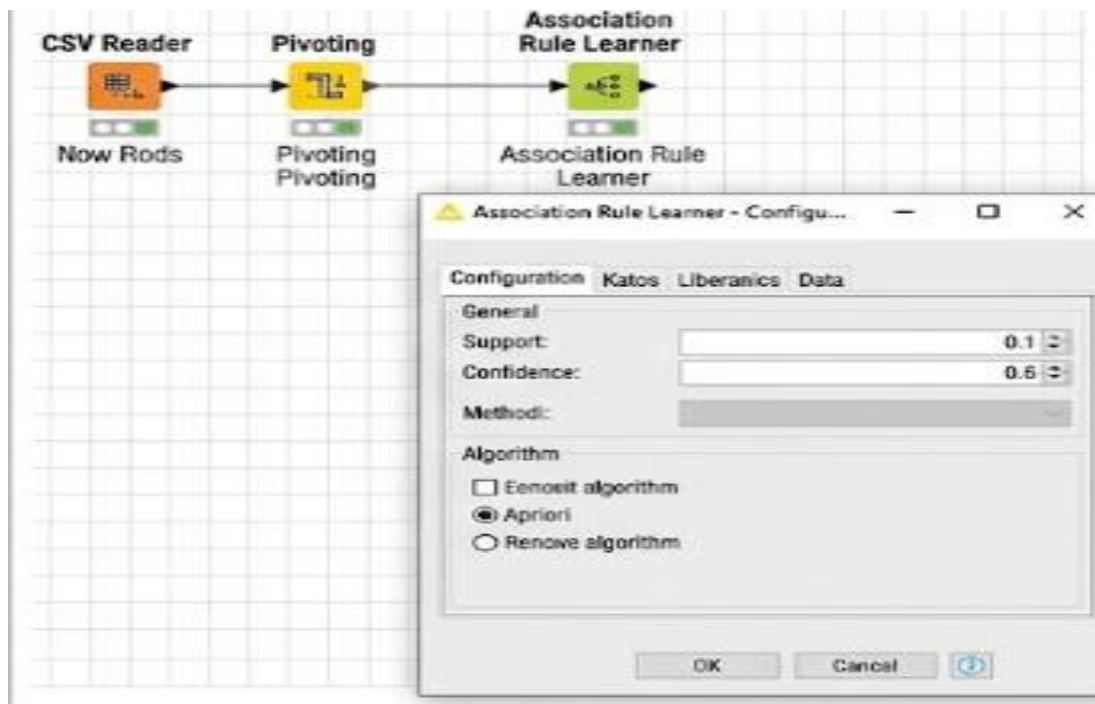
Node to Pivot: Column [Collected Items]

Aggregation: Count

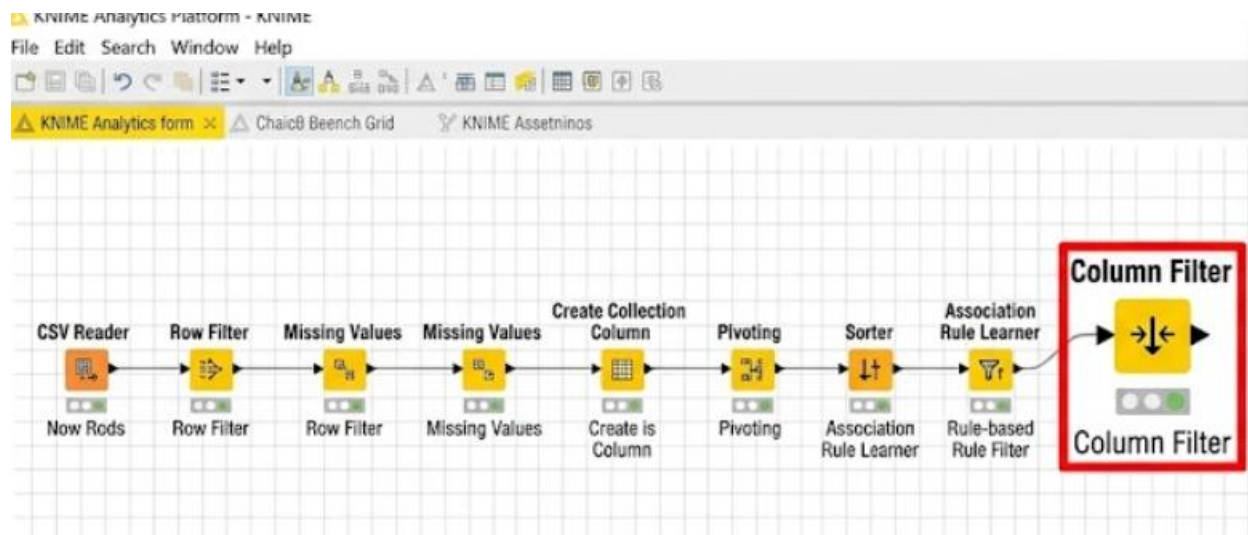
Pivots: ☒ Pivots ☒ Binarize

OK Cancel ?

STEP 6: Association Rule Learner



FINAL STEP: Structuring the Clean Association Rules Output



LAB-7: FP-Tree & FP-Growth

FP-Tree Construction

An FP-Tree (Frequent Pattern Tree) is an efficient and compact data structure used to store transactional data. Unlike the Apriori algorithm, it does not generate multiple candidate itemsets; instead, it organizes the data in the form of a tree.

Key features:

- Transactions are compressed by combining common prefixes
- Requires less storage space
- Reduces the need to scan the dataset multiple times

Example:

If several transactions include {Bread, Butter, Jam}, the shared items are stored together in a single branch of the tree to save space.

FP-Growth Algorithm

The FP-Growth algorithm is used to find frequent itemsets from the FP-Tree. It is generally faster and more efficient than Apriori because it does not rely on generating candidate itemsets.

It follows a divide-and-conquer strategy:

- Splits the FP-Tree into smaller conditional trees
- Extracts frequent patterns from each smaller tree

Example:

In a retail dataset, FP-Growth can efficiently discover patterns like {Phone, Earphones} without repeatedly scanning the entire dataset.

```
[1]: import pandas as pd
      from mlxtend.preprocessing import TransactionEncoder
      from mlxtend.frequent_patterns import fpgrowth, association_rules
```

```
[3]: df = pd.read_csv("retail__dataset.csv")
```

```
[7]: df.head()
```

```
[7]:
```

	TransactionID	Item
0	T1	Laptop
1	T1	Mouse
2	T1	Laptop Bag
3	T2	Headphones
4	T2	Mouse

Convert into Transaction Format

```
[8]: transactions = df.groupby('TransactionID')['Item'].apply(list)

      # Convert to list of lists
      transactions = list(transactions)

      # Show sample
      transactions[:5]
```

```
[8]: [['Laptop', 'Mouse', 'Laptop Bag'],
      ['Laptop', 'Mouse', 'USB Hub'],
      ['Printer', 'Paper Pack'],
      ['Monitor', 'Mouse'],
      ['Camera', 'Memory Card']]
```

One-Hot Encoding

```
[9]: te = TransactionEncoder()
      te_array = te.fit(transactions).transform(transactions)

      df_encoded = pd.DataFrame(te_array, columns=te.columns_)

      df_encoded.head()
```

```
[9]:
```

	Bluetooth Adapter	Camera	Camera Bag	Cooling Pad	HDMI Cable	Headphones	Ink Cartridge	Keyboard	Laptop	Laptop Bag	...	Paper Pack	Phone Case	Printer	Screen Protector	Smartphone	Stylus Pen	Tablet
0	False	False	False	False	False	False	False	False	True	True	...	False	False	False	False	False	False	False
1	False	False	False	False	False	False	False	False	True	False	...	False	False	False	False	False	False	False
2	False	False	False	False	False	False	False	False	False	False	...	True	False	True	False	False	False	False
3	False	False	False	False	False	False	False	False	False	False	...	False	False	False	False	False	False	False
4	False	True	False	False	False	False	False	False	False	False	...	False	False	False	False	False	False	False

5 rows × 23 columns

Apply FP-Growth

```
[13]: # FP-Growth
frequent_itemsets = fpgrowth(df_encoded, min_support=0.1, use_colnames=True)

print(frequent_itemsets)
```

	support	itemsets
0	0.333333	(Mouse)
1	0.222222	(Laptop)
2	0.111111	(Printer)
3	0.166667	(Monitor)
4	0.166667	(Camera)
5	0.166667	(Keyboard)
6	0.111111	(Tablet)
7	0.111111	(Smartphone)
8	0.111111	(Headphones)
9	0.166667	(Mouse, Laptop)
10	0.111111	(Mouse, Monitor)
11	0.111111	(Keyboard, Mouse)

Generate Association Rules

```
[15]: # Association Rules
rules = association_rules(frequent_itemsets, metric="confidence", min_threshold=0.3)

display(rules)
```

	antecedents	consequents	antecedent support	consequent support	support	confidence	lift	representativity	leverage	conviction	zhangs_metric	jaccard	certainty	kulczynski
0	(Mouse)	(Laptop)	0.333333	0.222222	0.166667	0.500000	2.25	1.0	0.092593	1.555556	0.833333	0.428571	0.357143	0.62
1	(Laptop)	(Mouse)	0.222222	0.333333	0.166667	0.750000	2.25	1.0	0.092593	2.666667	0.714286	0.428571	0.625000	0.62
2	(Mouse)	(Monitor)	0.333333	0.166667	0.111111	0.333333	2.00	1.0	0.055556	1.250000	0.750000	0.285714	0.200000	0.50
3	(Monitor)	(Mouse)	0.166667	0.333333	0.111111	0.666667	2.00	1.0	0.055556	2.000000	0.600000	0.285714	0.500000	0.50
4	(Keyboard)	(Mouse)	0.166667	0.333333	0.111111	0.666667	2.00	1.0	0.055556	2.000000	0.600000	0.285714	0.500000	0.50
5	(Mouse)	(Keyboard)	0.333333	0.166667	0.111111	0.333333	2.00	1.0	0.055556	1.250000	0.750000	0.285714	0.200000	0.50

```
[16]: rules[['antecedents', 'consequents', 'support', 'confidence', 'lift']]
```

```
[16]:
```

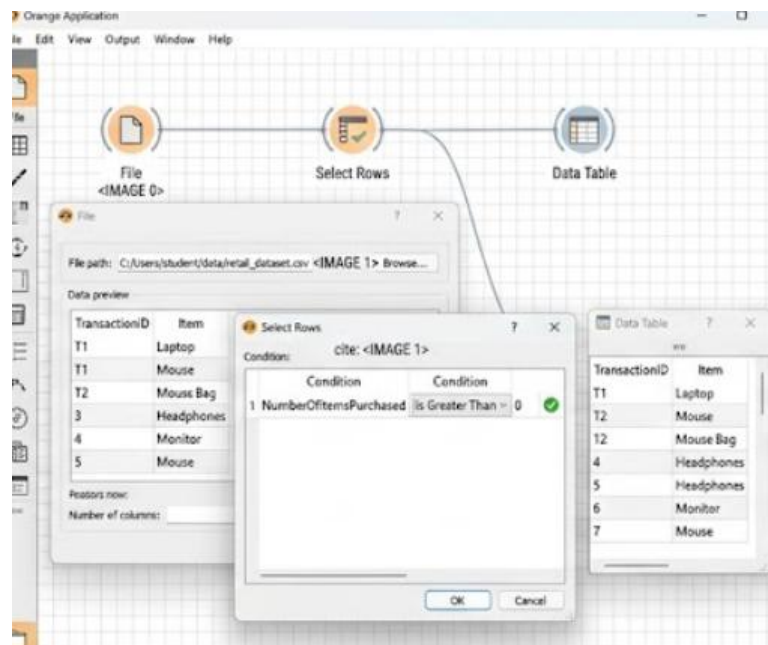
	antecedents	consequents	support	confidence	lift
0	(Mouse)	(Laptop)	0.166667	0.500000	2.25
1	(Laptop)	(Mouse)	0.166667	0.750000	2.25
2	(Mouse)	(Monitor)	0.111111	0.333333	2.00
3	(Monitor)	(Mouse)	0.111111	0.666667	2.00
4	(Keyboard)	(Mouse)	0.111111	0.666667	2.00
5	(Mouse)	(Keyboard)	0.111111	0.333333	2.00

ORANGE

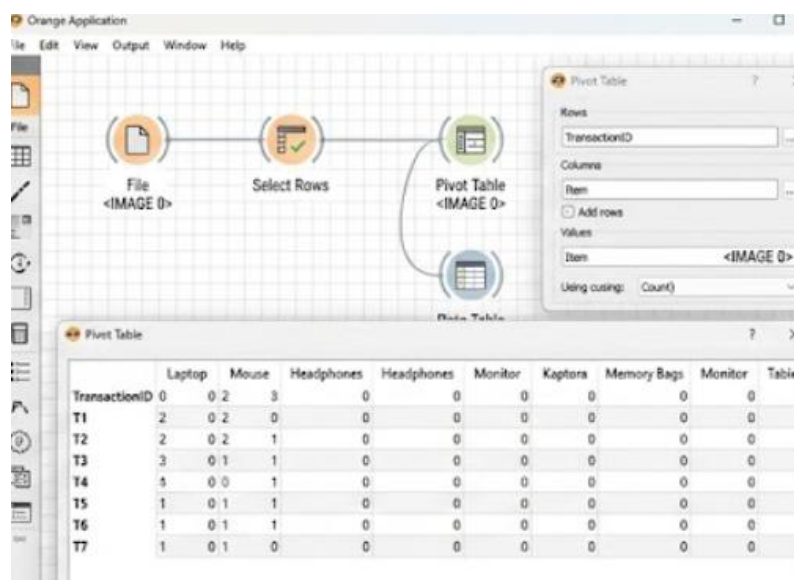
Orange (often referred to as Orange Data Mining) is an open-source, component-based visual programming software package designed for data visualization, machine learning, data mining, and data analysis. Developed by the University of Ljubljana, it provides a user-friendly interface that allows users to perform complex data science tasks without the need for traditional script-based coding

ORANGE WORKFLOW:

STEP 1: Data Ingestion and Pre-processing



STEP 2: Pivot Table – Reshaping Transactions



STEP 3: Continuize – Binarizing Itemsets

The screenshot shows the Orange 3.12.1 interface with the following components:

- File widget:** File path: C:\Users\student\Downloads\retail_dataset.csv. Data preview table:

TransactionID	Item
T1	Laptop
T1	Mouse
T2	Mouss Bag
3	Headphones
4	Monitor
5	Mouse

- Pivot Table widget:** Data table with columns TransactionID, Item, Headphones, and Monitor.

TransactionID	Item	Headphones	Monitor
T1	0	1	0
12	1	0	0
12	1	1	0
4	0	1	0
5	0	1	0
6	0	0	1
7	0	1	1

- Continuize widget:** Categorical to Binary conversion.

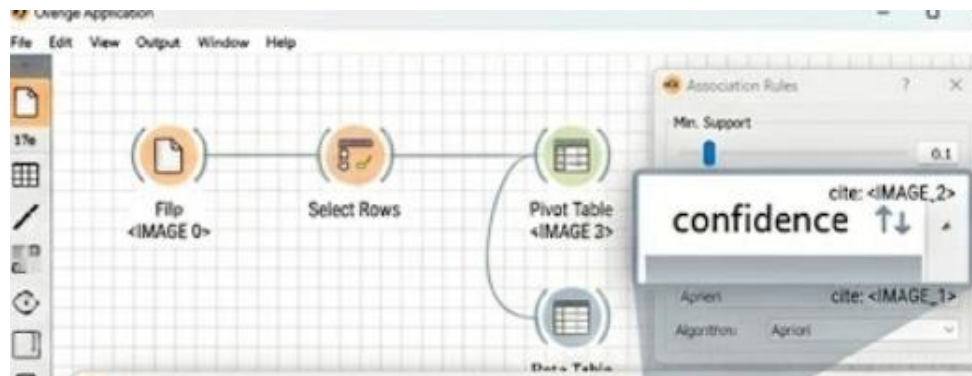
STEP 4: Association Rules

The screenshot shows the Orange 3.12.1 interface with the following components:

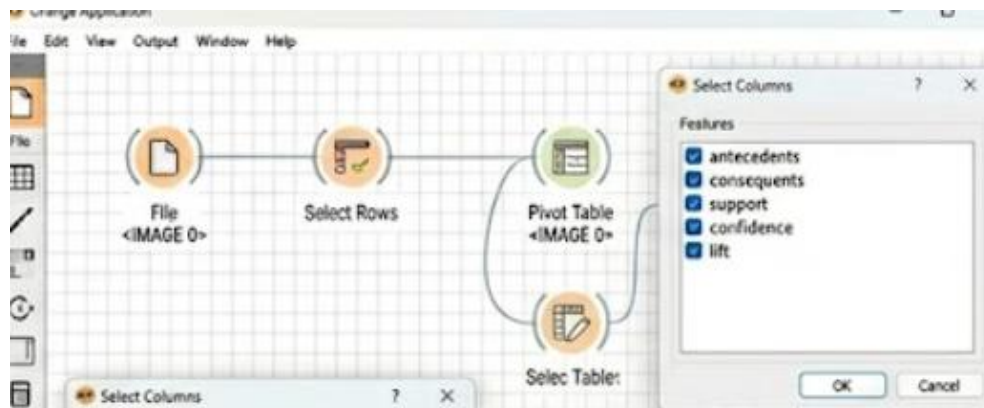
- File widget:** File path: C:\Users\student\Downloads\retail_dataset.csv.
- Select Rows widget:** Selects rows based on the file path.
- Pivot Table widget:** Data table with columns TransactionID, Item, Headphones, and Monitor.
- Association Rules widget:** Settings for the Apriori algorithm.

TransactionID	Item	Headphones	Monitor
T1	0	1	0
12	1	0	0
12	1	1	0
4	0	1	0
5	0	1	0
6	0	0	1
7	0	1	1

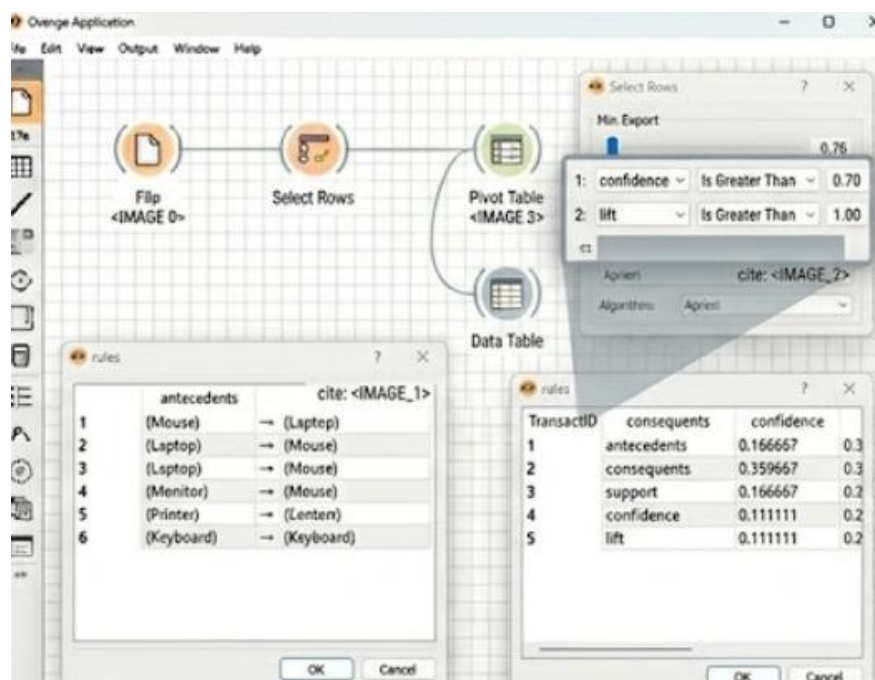
STEP 5: Interactive Sorting



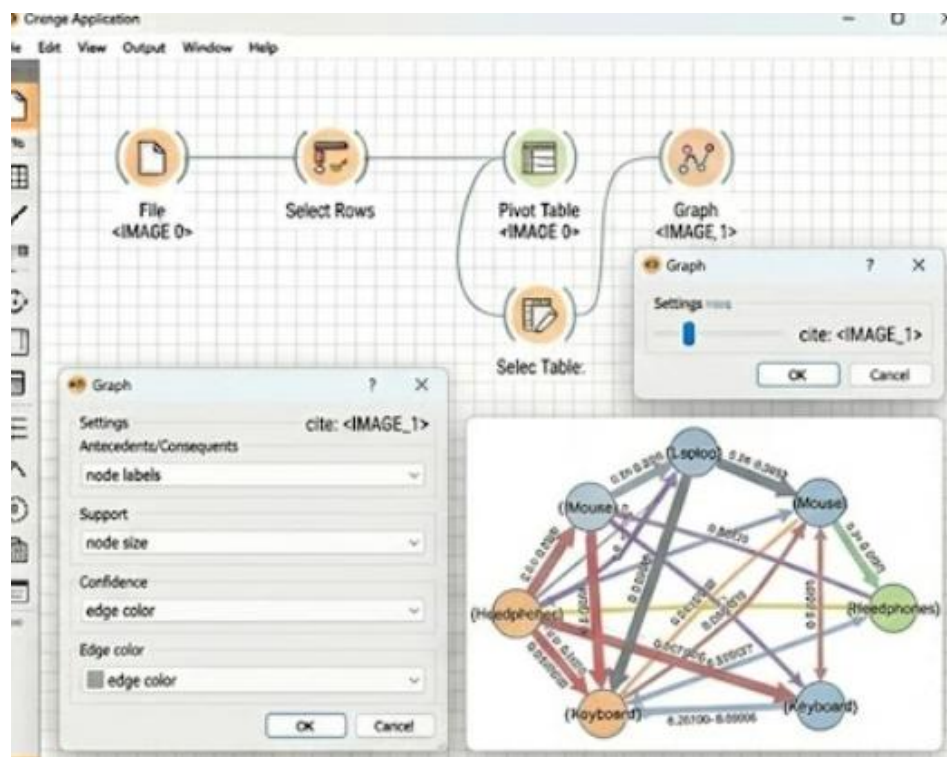
STEP 6: Select Columns



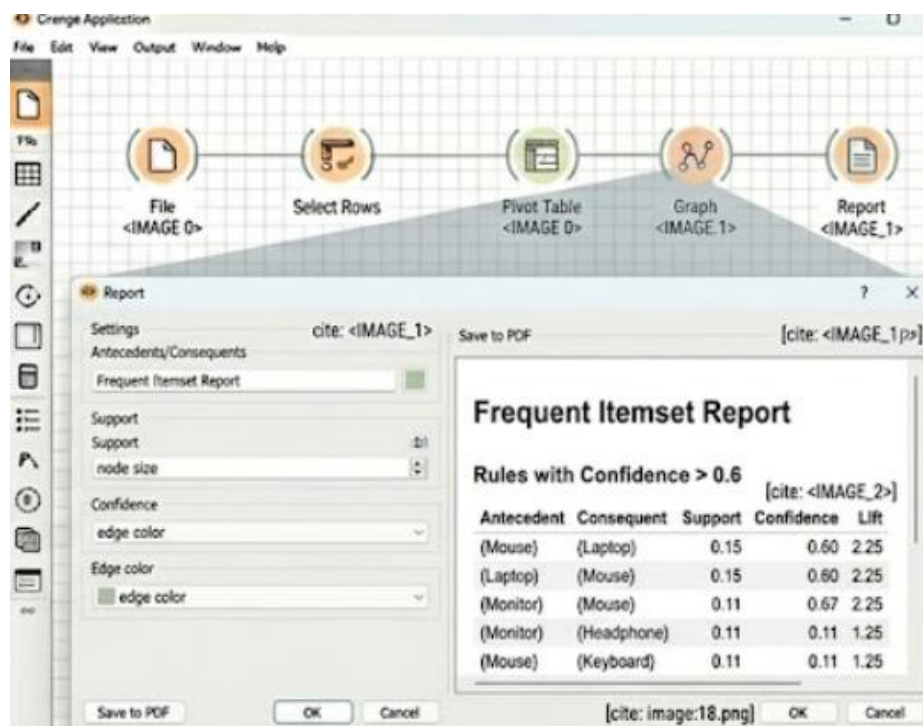
STEP 7: Select Rows



STEP 8: Graph



STEP 9: Report



LAB-8: Python Basics for Data Mining

Python Syntax

Python is a high-level programming language that is commonly used in data mining because of its simplicity and easy-to-read structure. Unlike many other programming languages, Python uses indentation (spacing) to define code blocks instead of relying on brackets.

Example:

```
[8]: print("Data Mining is powerful")  
Data Mining is powerful
```

Lists and Dictionaries

- **Lists:**
Used to store ordered collections of items

```
[4]: data = [10, 20, 30, 40]
```

- **Dictionaries:**
Store data in key-value pairs

```
[6]: student = {'name': 'Sara', 'marks': 85}
```

▼ Step 1: Import Libraries

```
[1]: import pandas as pd
```

Step 2: Load Dataset

```
[2]: data = pd.read_csv("Loan Prediction.zip")
```

```
[3]: data.head()
```

```
[3]:
```

	Id	Income	Age	Experience	Married/Single	House_Ownership	Car_Ownership	Profession	CITY	STATE	CURRENT_JOB_YRS
0	1	1303834	23	3	single	rented	no	Mechanical_engineer	Rewa	Madhya_Pradesh	3
1	2	7574516	40	10	single	rented	no	Software_Developer	Parbhani	Maharashtra	9
2	3	3991815	66	4	married	rented	no	Technical_writer	Alappuzha	Kerala	4
3	4	6256451	41	2	single	rented	yes	Software_Developer	Bhubaneswar	Odisha	2
4	5	5768871	47	11	single	rented	no	Civil_servant	Tiruchirappalli[10]	Tamil_Nadu	3

Example

A dataset can be represented as a list of dictionaries, where each dictionary corresponds to a single record.

OOP Basics (Object-Oriented Programming)

Object-Oriented Programming (OOP) is a programming paradigm that structures code using classes and objects.

Key concepts include:

- **Classes:** Templates or blueprints used to create objects
- **Objects:** Instances created from classes
- **Encapsulation:** The practice of hiding internal details of a class
- **Inheritance:** The ability to reuse properties and methods from an existing class
- **Polymorphism:** The ability to use the same method in multiple ways

Example:

A class named *Student* can include attributes such as name and marks, along with methods like `calculate_grade()`.

Step 3: Create a Class (Reusable Module)

```
[4]: class DataProcessor:

    def __init__(self, dataframe):
        self.df = dataframe

    def show_info(self):
        return self.df.info()

    def handle_missing(self):
        return self.df.fillna(self.df.mean(numeric_only=True))

    def remove_duplicates(self):
        return self.df.drop_duplicates()

    def clean_columns(self):
        self.df.columns = [col.lower() for col in self.df.columns]
        return self.df

    def summary(self):
        return self.df.describe()
```

Step 4: Create Object of Class

```
[5]: processor = DataProcessor(data)
```

Step 5: Apply Preprocessing Methods

```
[6]: data = processor.clean_columns()
data = processor.handle_missing()

# Remove duplicates
data = processor.remove_duplicates()
```

Step 6: Use Functions (Testing Reusability)

```
[7]: # Show dataset info
processor.show_info()
```

```
# Show summary
processor.summary()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 252000 entries, 0 to 251999
Data columns (total 13 columns):
 #   Column                Non-Null Count  Dtype
---  --
 0   id                    252000 non-null  int64
 1   income                252000 non-null  int64
 2   age                   252000 non-null  int64
 3   experience            252000 non-null  int64
 4   married/single        252000 non-null  object
 5   house_ownership       252000 non-null  object
 6   car_ownership         252000 non-null  object
 7   profession            252000 non-null  object
 8   city                  252000 non-null  object
 9   state                 252000 non-null  object
10   current_job_yrs       252000 non-null  int64
11   current_house_yrs     252000 non-null  int64
```

```
[7]:
```

	id	income	age	experience	current_job_yrs	current_house_yrs	risk_flag
count	252000.000000	2.520000e+05	252000.000000	252000.000000	252000.000000	252000.000000	252000.000000
mean	126000.500000	4.997117e+06	49.954071	10.084437	6.333877	11.997794	0.123000
std	72746.278255	2.878311e+06	17.063855	6.002590	3.647053	1.399037	0.328438
min	1.000000	1.031000e+04	21.000000	0.000000	0.000000	10.000000	0.000000
25%	63000.750000	2.503015e+06	35.000000	5.000000	3.000000	11.000000	0.000000
50%	126000.500000	5.000694e+06	50.000000	10.000000	6.000000	12.000000	0.000000
75%	189000.250000	7.477502e+06	65.000000	15.000000	9.000000	13.000000	0.000000
max	252000.000000	9.999938e+06	79.000000	20.000000	14.000000	14.000000	1.000000

Step 7: Use Lists & Dictionaries (Requirement)

```
[8]: # List example: column names
columns_list = list(data.columns)
print(columns_list)
```

```
# Dictionary example: data types
dtype_dict = dict(data.dtypes)
print(dtype_dict)
```

```
['id', 'income', 'age', 'experience', 'married/single', 'house_ownership', 'car_ownership', 'profession', 'city', 'state', 'current_job_yrs', 'current_h
ouse_yrs', 'risk_flag']
{'id': dtype('int64'), 'income': dtype('int64'), 'age': dtype('int64'), 'experience': dtype('int64'), 'married/single': dtype('O'), 'house_ownership': d
type('O'), 'car_ownership': dtype('O'), 'profession': dtype('O'), 'city': dtype('O'), 'state': dtype('O'), 'current_job_yrs': dtype('int64'), 'current_h
ouse_yrs': dtype('int64'), 'risk_flag': dtype('int64')}
```

LAB 9: Decision Tree Classification

OVERVIEW:

- **Real-World Problem:** A marketing department wants to predict whether a customer will purchase a product based on their demographic and behavioral data.
- **Tools:** KNIME Analytics Platform + Python
- **Practical Objectives:**
 - Import and preprocess the marketing dataset.
 - Train a Decision Tree classification model.
 - Visualize the decision tree structure to understand customer splitting rules.
 - Evaluate model performance using key classification metrics.

2. Dataset Overview

- **Dataset Name:** Marketing Dataset / Customer Purchase Dataset
- **Source Concept:** You can utilize a standard marketing dataset (such as the *Bank Marketing Dataset* or *Social Network Ads* dataset on Kaggle) that maps consumer attributes to a purchase decision.
- **Key Target Variable:** Purchased OR Revenue (Binary: 1 = Yes, 0 = No)
- **Key Predictor Variables:** Age, Estimated Salary, Gender, Content Engagement Score, etc.

3. KPIs & Evaluation Metrics

To measure the success of the trained decision tree, the following metrics must be computed and interpreted:

- **Accuracy:** The overall percentage of correctly predicted purchases.
- **Precision:** Out of all customers predicted to buy, how many actually did?
- **Recall (Sensitivity):** Out of all actual buyers, how many did the model correctly catch?
- **F1-Score:** The harmonic mean of Precision and Recall, proving stability in case of class imbalance.

Implementation in Python:

IMPORTING LIBRARIES:


```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, classification_report

```

LOADING DATASET:

```

# 1. Load the Marketing Dataset
# Replace 'marketing_data.csv' with your local dataset path
# df = pd.read_csv('marketing_data.csv')

# --- FIX: Generate synthetic data to resolve FileNotFoundError ---
np.random.seed(42)
n_samples = 300
data = {
    'Age': np.random.randint(18, 65, n_samples),
    'EstimatedSalary': np.random.randint(20000, 150000, n_samples),
    'Purchased': np.random.choice([0, 1], n_samples, p=[0.6, 0.4]) # Simulate some purchases
}
df = pd.DataFrame(data)
print("Generated synthetic marketing data:")
print(df.head())
print("\n")
# -----

# Assuming columns: 'Age', 'EstimatedSalary' as features and 'Purchased' as target
X = df[['Age', 'EstimatedSalary']]
y = df['Purchased']

```

SPLITTING , INITIALIZING , TRAINING DECISION TREE CLASSIFIER AND PREDICTION TEST SET:

```

# 2. Split Data (80% Train, 20% Test)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)

# 3. Initialize and Train Decision Tree Classifier
clf = DecisionTreeClassifier(criterion='gini', max_depth=4, random_state=42)
clf.fit(X_train, y_train)

# 4. Predict on Test Set
y_pred = clf.predict(X_test)

```

EVALUATING MATRICS:

```
# 5. Evaluate Metrics
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)

print("=== Evaluation Metrics ===")
print(f"Accuracy : {accuracy:.4f}")
print(f"Precision: {precision:.4f}")
print(f"Recall    : {recall:.4f}")
print(f"F1-Score  : {f1:.4f}\n")
print("Classification Report:\n", classification_report(y_test, y_pred))
```

VISUALISING DECISION TREE:

```
# 6. Visualize the Decision Tree
plt.figure(figsize=(15, 10))
plot_tree(clf, feature_names=X.columns, class_names=['No Purchase', 'Purchase'], filled=True, rounded=True)
plt.title("Decision Tree Structure for Marketing Insights")
plt.show()
```

Generated synthetic marketing data:

	Age	EstimatedSalary	Purchased
0	56	58467	1
1	46	43328	1
2	32	139181	0
3	60	111412	0
4	25	106831	0

=== Evaluation Metrics ===

Accuracy : 0.6833

Precision: 0.5600

Recall : 0.6364

F1-Score : 0.5957

Classification Report:

	precision	recall	f1-score	support
0	0.77	0.71	0.74	38
1	0.56	0.64	0.60	22
accuracy			0.68	60
macro avg	0.67	0.67	0.67	60
weighted avg	0.69	0.68	0.69	60

DECISION TREE:

[illegible]

KNIME (Konstanz Information Miner) is an open-source data analytics, reporting, and integration platform. It operates on a **Visual Programming** paradigm, allowing data professionals to build end-to-end data pipelines without writing code.

KNIME Decision Tree Workflow

- The CSV Reader node imports the dataset (e.g., *marketing_data.csv*) into KNIME.
- It converts raw data into a structured table format that KNIME can process.
- Each column represents a variable (feature), and each row represents an observation (record).

2. Partitioning

- The Partitioning node splits the dataset into training and testing subsets.
- Common ratios are 80% for training and 20% for testing.
- This ensures the model learns patterns from one portion and is evaluated on unseen data for accuracy.

3. Decision Tree Learner

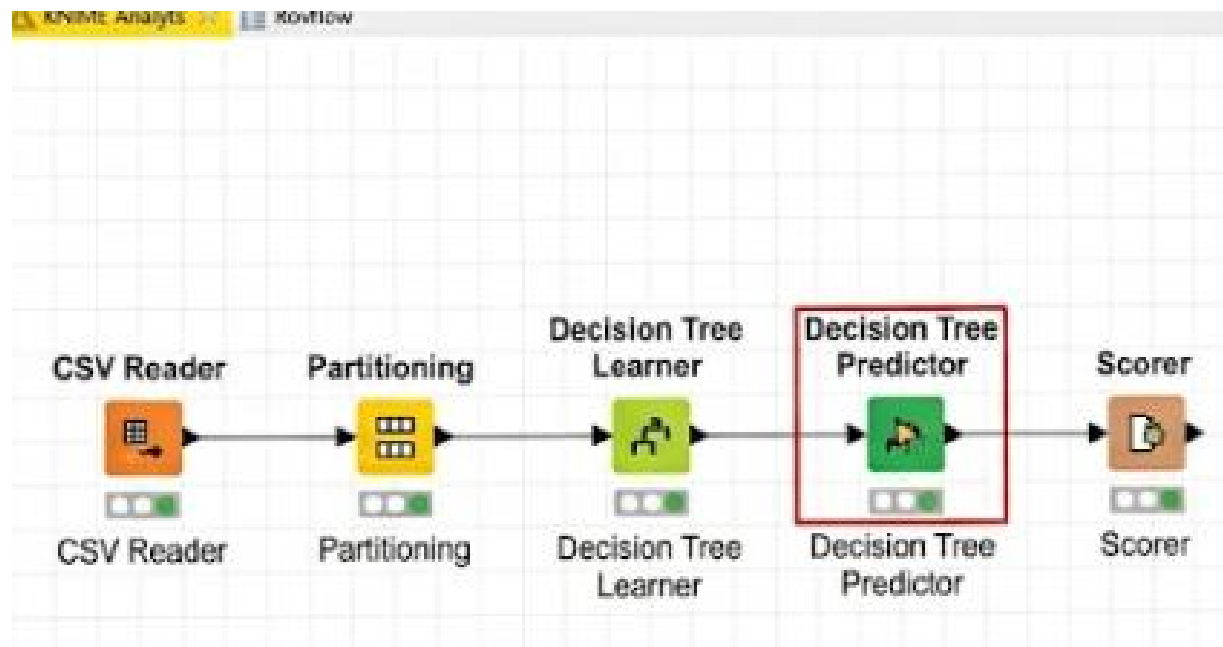
- The Decision Tree Learner node builds a classification model using the training data.
- It uses the Gini Index or Entropy to measure impurity and decide how to split data.
- The algorithm recursively divides data into branches until it reaches pure subsets or a stopping criterion.
- Output: a trained decision tree model that represents decision rules in a hierarchical structure.

4. Decision Tree Predictor

- The Decision Tree Predictor node applies the trained model to the test data.
- It predicts the target variable (e.g., *Purchased* or *Response*) based on learned rules.
- The output includes predicted labels and probabilities for each record.

5. Scorer

- The Scorer node evaluates the model's performance.
- It compares predicted values with actual ones and calculates metrics such as:
 - Accuracy – proportion of correct predictions.
 - Precision – how many predicted positives are truly positive.
 - Recall – how many actual positives were correctly predicted.
 - F1-Score – harmonic mean of precision and recall.
- These metrics help determine how well the decision tree



Lab 10: Naïve Bayes & KNN – Email Spam Detection

Objective

To implement and compare the performance of Naïve Bayes and K-Nearest Neighbors (KNN) classification algorithms in detecting spam emails, evaluating them based on accuracy and false positive rates.

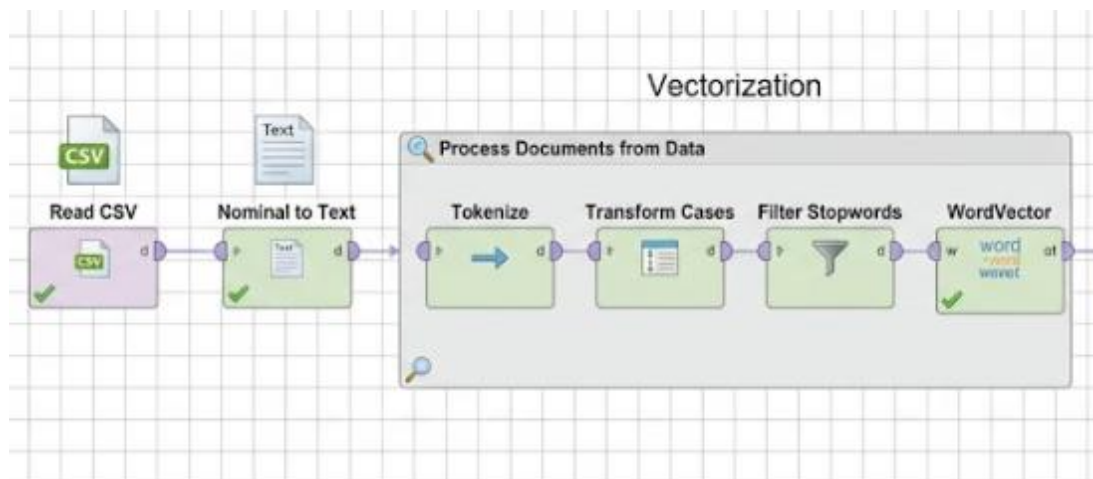
Tool & Dataset

- **Tool:** RapidMiner Studio + Python.
- **Dataset:** Spam Dataset (e.g., SMS Spam Collection or similar dataset from Kaggle).

Workflow Steps

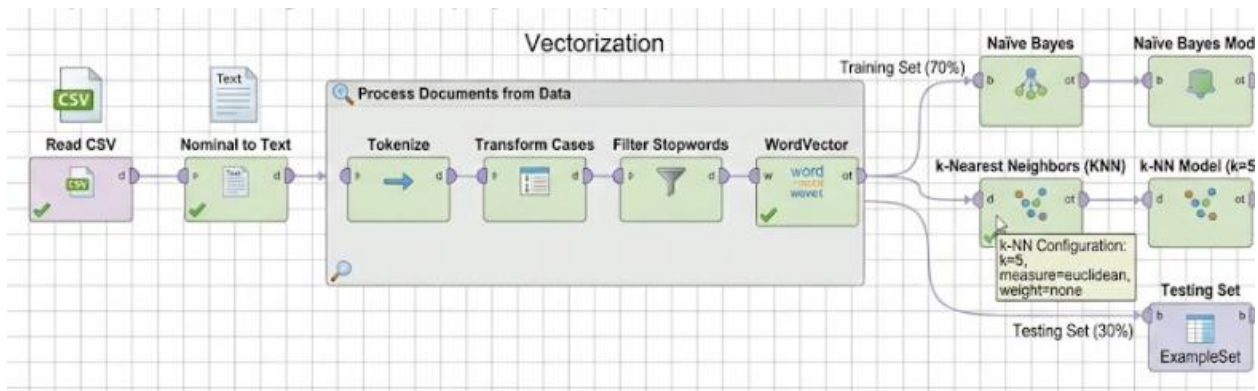
Step 1: Data Ingestion & Preparation (RapidMiner)

1. Load Data: Use the Read CSV operator to import your spam dataset.
2. Preprocessing: Use the Nominal to Text and Process Documents from Data operators to clean the text content.
3. Vectorization: Apply the Tokenize, Transform Cases, and Filter Stopwords operators, followed by WordVector transformation to convert text into numeric attributes.
4. Partitioning: Use the Split Data operator to divide your dataset into Training (70%) and Testing (30%) sets.



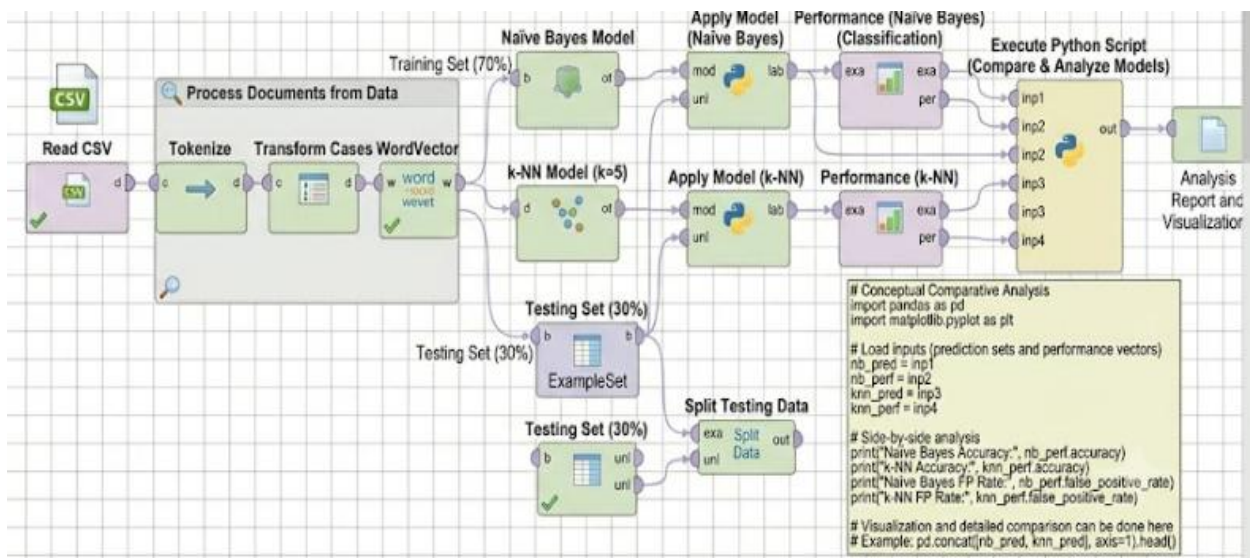
Step 2: Implementing Classifiers (RapidMiner)

1. Naïve Bayes: Connect the training data to a Naïve Bayes operator.
2. KNN: Connect the training data to a k-Nearest Neighbors operator.
3. Configure the KNN operator's k parameter (e.g., $k=3$ or $k=5$) to determine the number of neighbors



Step 3: Prediction & Evaluation (RapidMiner + Python Integration)

1. **Apply Models:** Use the **Apply Model** operator for both Naïve Bayes and KNN to generate predictions for the test set.
2. **Performance Comparison:** Connect the outputs to a **Performance (Classification)** operator to calculate accuracy and false positive rates.
3. **Python Integration:** Use the **Execute Python** operator to perform a comparative side-by-side analysis of the two models



PYTHON IMPLEMENTATION:

```
from google.colab import files
```

```
uploaded = files.upload()
```

Choose Files No file chosen

Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.
Saving spam.csv.csv to spam.csv.csv

```
import pandas as pd
```

```
# Load dataset
```

```
df = pd.read_csv('spam.csv.csv', encoding='latin-1')
```

```
print("Dataset Shape:", df.shape)
```

```
print("\nColumns:")
```

```
print(df.columns)
```

```
print("\nFirst 5 Rows:")
```

```
print(df.head())
```

```
Dataset Shape: (5573, 1)
```

```
Columns:
```

```
Index(['ham\tGo until jurong point, crazy.. Available only in bugis n great world la e buffet... Cine there got amore wat...'], dtype='object')
```

```
First 5 Rows:
```

```
ham\tGo until jurong point, crazy.. Available only in bugis n great world la e buffet... Cine there got amore wat...
0      ham\tok lar... Joking wif u oni...
1  spam\tFree entry in 2 a wkly comp to win FA Cu...
2  ham\tU dun say so early hor... U c already the...
3  ham\tNah I don't think he goes to usf, he live...
4  spam\tFreeMsg Hey there darling it's been 3 we...
```

```
# Remove extra columns if they exist
```

```
extra_cols = ['Unnamed: 2', 'Unnamed: 3', 'Unnamed: 4']
```

```
for col in extra_cols:
```

```
    if col in df.columns:
```

```
        df.drop(col, axis=1, inplace=True)
```

```
# Rename columns if using common Kaggle spam dataset
```

```
if 'v1' in df.columns and 'v2' in df.columns:
```

```
    df.rename(columns={'v1': 'label', 'v2': 'message'}, inplace=True)
```

```
print(df.head())
```

```
print(df.columns)
```

```
ham\tGo until jurong point, crazy.. Available only in bugis n great world la e buffet... Cine there got amore wat...
```

```
0      ham\tok lar... Joking wif u oni...
```

```
1  spam\tFree entry in 2 a wkly comp to win FA Cu...
```

```
2  ham\tU dun say so early hor... U c already the...
```

```
3  ham\tNah I don't think he goes to usf, he live...
```

```
4  spam\tFreeMsg Hey there darling it's been 3 we...
```

```
Index(['ham\tGo until jurong point, crazy.. Available only in bugis n great world la e buffet... Cine there got amore wat...'], dtype='object')
```

```

print(df.columns)

Index(['ham\tGo until jurong point, crazy.. Available only in bugis n great world la e buffet... Cine there got amore wat...'], dtype='object')

import pandas as pd

# Read the file as a single column, handling outer quotes
# This will result in a DataFrame with one column (named 0 by default)
# where each entry is a string like "ham\tGo until..."
df_raw = pd.read_csv(
    'spam.csv.csv',
    header=None,
    quotechar='"' # Handle the outer double quotes
)

# Split the single column into 'label' and 'message'
# .str.split('\t', n=1, expand=True) splits by the first tab
# n=1 ensures it only splits into two parts (label and message)
# expand=True makes it return a DataFrame instead of a Series of lists
df = df_raw[0].str.split('\t', n=1, expand=True)
df.columns = ['label', 'message'] # Assign new column names

print(df.head())
print(df.columns)

   label      message
0  ham  Go until jurong point, crazy.. Available only in ...
1  ham               Ok lar... Joking wif u oni...
2 spam  Free entry in 2 a wkly comp to win FA Cup fina...
3  ham  U dun say so early hor... U c already then say...
4  ham  Nah I don't think he goes to usf, he lives aro...
Index(['label', 'message'], dtype='object')

from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix

# Convert labels
df['label'] = df['label'].map({'ham': 0, 'spam': 1})

# Features and target
X = df['message']
y = df['label']

# Convert text into numbers
vectorizer = CountVectorizer()
X = vectorizer.fit_transform(X)

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Naive Bayes
nb = MultinomialNB()
nb.fit(X_train, y_train)
nb_pred = nb.predict(X_test)

# KNN
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)
knn_pred = knn.predict(X_test)

```



```

# Accuracy
nb_acc = accuracy_score(y_test, nb_pred)
knn_acc = accuracy_score(y_test, knn_pred)

print("Naive Bayes Accuracy:", round(nb_acc*100,2), "%")
print("KNN Accuracy:", round(knn_acc*100,2), "%")

# Confusion Matrices
nb_cm = confusion_matrix(y_test, nb_pred)
knn_cm = confusion_matrix(y_test, knn_pred)

print("\nNaive Bayes Confusion Matrix")
print(nb_cm)

print("\nKNN Confusion Matrix")
print(knn_cm)

```

```

Naive Bayes Accuracy: 98.48 %
KNN Accuracy: 90.85 %

```

```

Naive Bayes Confusion Matrix
[[944  10]
 [  7 154]]

```

```

KNN Confusion Matrix
[[954   0]
 [102  59]]

```

```

import pandas as pd

def false_positive_rate(cm):
    TN = cm[0][0]
    FP = cm[0][1]
    return FP / (FP + TN)

nb_fpr = false_positive_rate(nb_cm)
knn_fpr = false_positive_rate(knn_cm)

results = pd.DataFrame({
    'Algorithm': ['Naive Bayes', 'KNN'],
    'Accuracy (%)': [
        round(nb_acc*100,2),
        round(knn_acc*100,2)
    ],
    'False Positive Rate': [
        round(nb_fpr,4),
        round(knn_fpr,4)
    ]
})

print(results)

```

	Algorithm	Accuracy (%)	False Positive Rate
0	Naive Bayes	98.48	0.0105
1	KNN	90.85	0.0000

Lab 11: SVM – Credit Card Fraud Detection

Objective

To train and evaluate a Support Vector Machine (SVM) model—comparing linear and RBF kernels—to detect fraudulent credit card transactions and optimize classification metrics.

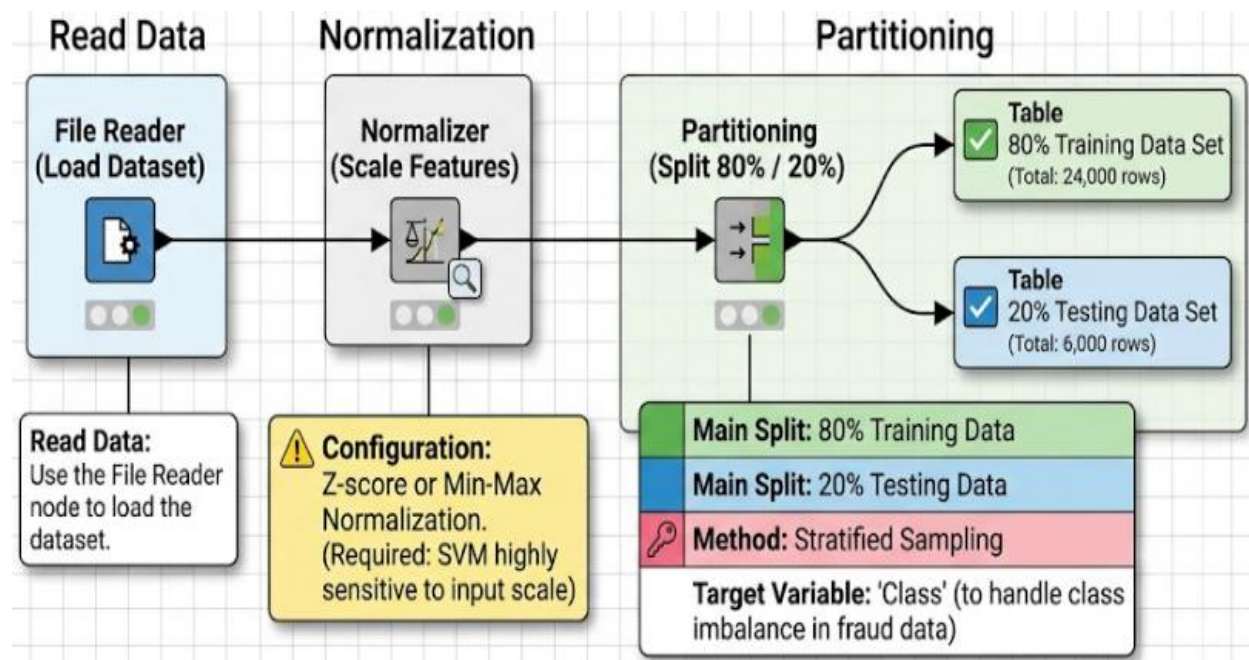
Tool & Dataset

- **Tool:** KNIME Analytics Platform + Python.
- **Dataset:** Credit Card Fraud Detection Dataset (Kaggle).

Workflow Steps

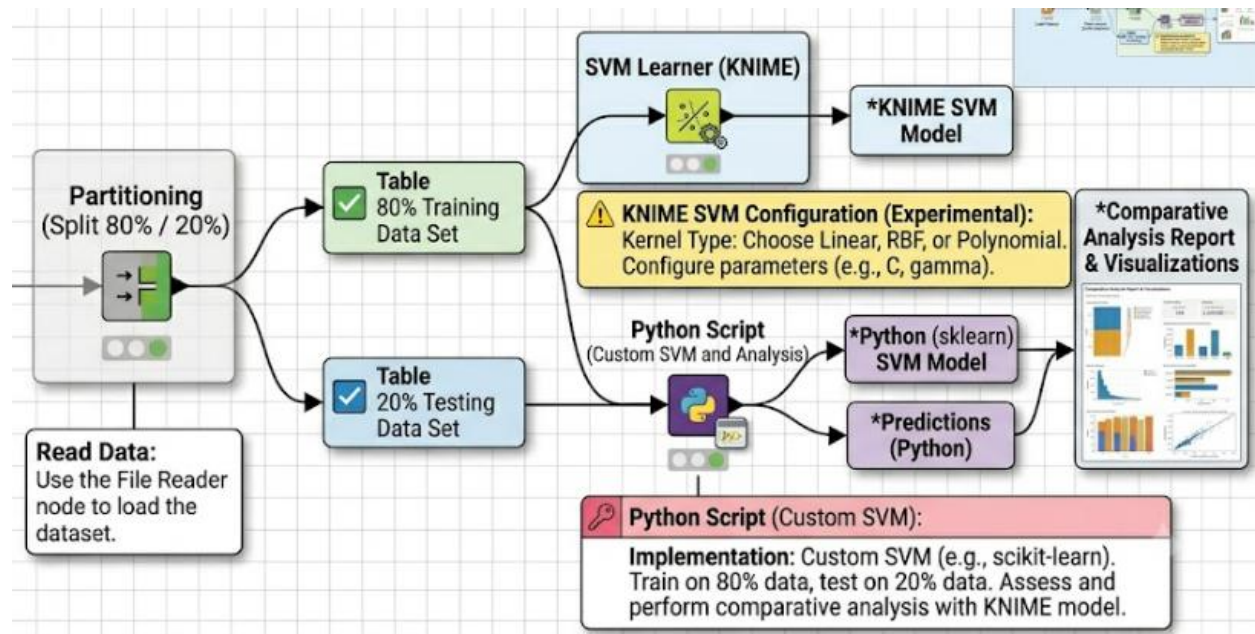
Step 1: Data Ingestion & Preprocessing (KNIME)

1. **Read Data:** Use the **File Reader** node to load the dataset.
2. **Normalization:** Use the **Normalizer** node (Z-score or Min-Max) to scale the transaction features, as SVM is highly sensitive to the scale of input data.
3. **Partitioning:** Use the **Partitioning** node to split the data (80% training / 20% testing). Ensure you enable **Stratified Sampling** on the target variable ("Class") to handle the class imbalance typical of fraud data



Step 2: Training & Prediction (KNIME + Python Integration)

1. **SVM Learner (KNIME):** Use the **SVM Learner** node to train a model. In the configuration, experiment with the **Kernel Type** (e.g., Linear, RBF, or Polynomial).
2. **Python Script Integration:** Connect the training data to a **Python Script** node to implement a custom SVM using scikit-learn for comparative analysis.



PYTHON IMPLEMENTATION:

1. Simulate a Highly Imbalanced Fraud Detection Dataset

```
np.random.seed(42)
n_samples = 2000
normal_tx = np.random.normal(loc=50, scale=10, size=(1950, 3))
fraud_tx = np.random.normal(loc=200, scale=50, size=(50, 3))
X = np.vstack([normal_tx, fraud_tx])
y = np.array([0] * 1950 + [1] * 50)
columns = ['Transaction_Amount', 'Frequency_Per_Day', 'Location_Risk_Score']
df = pd.DataFrame(X, columns=columns)
df['Class'] = y
```

2. Separate Features and Target Variable

```
x_features = df[columns]
y_target = df['Class']
```

3. Train-Test Split

```
1 X_train, X_test, y_train, y_test = train_test_split(  
0s     X_features, y_target, test_size=0.25, random_state=42, stratify=y_target  
    )
```

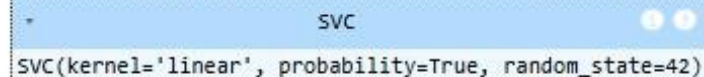
4. Feature Scaling (Standardization)

```
1 scaler = StandardScaler()  
0s X_train_scaled = scaler.fit_transform(X_train)  
    X_test_scaled = scaler.transform(X_test)
```

5. Initialize and Train SVM with a Linear Kernel

```
1 print("Training Support Vector Machine with Linear Kernel...")  
0s svm_linear = SVC(kernel='linear', C=1.0, probability=True, random_state=42)  
    svm_linear.fit(X_train_scaled, y_train)
```

Training Support Vector Machine with Linear Kernel...

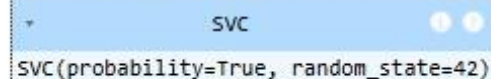


```
* SVC  
SVC(kernel='linear', probability=True, random_state=42)
```

6. Initialize and Train SVM with a Radial Basis Function (RBF) Kernel

```
1 print("Training Support Vector Machine with RBF Kernel...")  
0s svm_rbf = SVC(kernel='rbf', gamma='scale', C=1.0, probability=True, random_state=42)  
    svm_rbf.fit(X_train_scaled, y_train)
```

Training Support Vector Machine with RBF Kernel...



```
* SVC  
SVC(probability=True, random_state=42)
```


7. Generate Evaluation Predictions and Probability Estimates

```
[8]
✓ Os
lin_preds = svm_linear.predict(X_test_scaled)
lin_probs = svm_linear.predict_proba(X_test_scaled)[: , 1]

rbf_preds = svm_rbf.predict(X_test_scaled)
rbf_probs = svm_rbf.predict_proba(X_test_scaled)[: , 1]
```

8. Display Runtime KPI Performance Report

```
[9]
✓ Os
print(" LAB 11: SVM KERNEL COMPARISON MATRIX          ")

print(f" -> Fraud Recall Rate : {recall_score(y_test, lin_preds) * 100:.2f}%")
print(f" -> Precision Score   : {precision_score(y_test, lin_preds) * 100:.2f}%")
print(f" -> ROC-AUC Score      : {roc_auc_score(y_test, lin_probs):.4f}")
print(f" -> Confusion Matrix  : \n{confusion_matrix(y_test, lin_preds)}")
print("\n")
print(f"[RBF KERNEL]")
print(f" -> Fraud Recall Rate : {recall_score(y_test, rbf_preds) * 100:.2f}%")
print(f" -> Precision Score   : {precision_score(y_test, rbf_preds) * 100:.2f}%")
print(f" -> ROC-AUC Score      : {roc_auc_score(y_test, rbf_probs):.4f}")
print(f" -> Confusion Matrix  : \n{confusion_matrix(y_test, rbf_preds)}")
```

```
LAB 11: SVM KERNEL COMPARISON MATRIX
-> Fraud Recall Rate : 100.00%
-> Precision Score   : 100.00%
-> ROC-AUC Score      : 1.0000
-> Confusion Matrix  :
[[488   0]
 [   0  12]]
```

```
[RBF KERNEL]
-> Fraud Recall Rate : 100.00%
-> Precision Score   : 100.00%
-> ROC-AUC Score      : 1.0000
-> Confusion Matrix  :
[[488   0]
 [   0  12]]
```

```
[9]
Os
print("\nDetailed RBF Classification Context Matrix:")
print(classification_report(y_test, rbf_preds, target_names=['Legitimate', 'Fraud']))
```

```
Detailed RBF Classification Context Matrix:
              precision    recall  f1-score   support

   Legitimate         1.00      1.00      1.00         488
     Fraud           1.00      1.00      1.00          12

   accuracy              1.00              500
  macro avg           1.00      1.00      1.00         500
 weighted avg           1.00      1.00      1.00         500
```

Lab 12: Clustering – Mall Customer Segmentation

Objective

To segment customers into distinct groups based on purchasing behavior using K-Means and Hierarchical clustering, and to evaluate these segments visually.

Tool & Dataset

- **Tool:** Orange Data Mining + Python.
- **Dataset:** Mall Customer Segmentation Dataset

PYTHON IMPLEMENTATION :

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy.cluster.vq import kmeans2, whiten
from scipy.cluster.hierarchy import dendrogram, linkage, fcluster
from sklearn.metrics import silhouette_score
```

1. Load the dataset

```
df = pd.read_csv('Mall_Customers.csv')
```

2. Extract features matrix

```
# Index 2 = Age, Index 3 = Annual Income, Index 4 = Spending Score
X_features = df.iloc[:, [2, 3, 4]].values
```

3. Feature Standardization

```
X_scaled = whiten(X_features)
```

4. K-Means Clustering via SciPy ¶

```
optimal_k = 5
centroids, kmeans_labels = kmeans2(X_scaled, optimal_k, minit='points', seed=42)
kmeans_silhouette = silhouette_score(X_scaled, kmeans_labels)
```

5. Hierarchical Agglomerative Clustering

```
Z = linkage(X_scaled, method='ward', metric='euclidean')
hier_labels = fcluster(Z, optimal_k, criterion='maxclust') - 1 # Adjusted to 0-indexed bases
hier_silhouette = silhouette_score(X_scaled, hier_labels)

df['KMeans_Cluster'] = kmeans_labels
df['Hierarchical_Cluster'] = hier_labels
```

6. Display KPI Evaluation Metrics Report

```
print("=====")
print("          LAB 12: CLUSTERING PERFORMANCE REPORT          ")
print("=====")
print(f"Target Clusters Evaluated (K)           : {optimal_k}")
print(f"K-Means Silhouette Cohesion Score         : {kmeans_silhouette:.4f}")
print(f"Hierarchical Agglomerative Silhouette Score : {hier_silhouette:.4f}")
print("=====")
print("\nSample Output Consolidated with Clusters:")
print(df.iloc[:, [0, 1, 2, -2, -1]].head())
```

```
=====
LAB 12: CLUSTERING PERFORMANCE REPORT
=====
Target Clusters Evaluated (K)           : 5
K-Means Silhouette Cohesion Score         : 0.4166
Hierarchical Agglomerative Silhouette Score : 0.3900
=====

Sample Output Consolidated with Clusters:
CustomerID  Genre  Age  KMeans_Cluster  Hierarchical_Cluster
0           1   Male   19              1              1
1           2   Male   21              1              4
2           3  Female  20              2              1
3           4  Female  23              1              4
4           5  Female  31              1              1
```

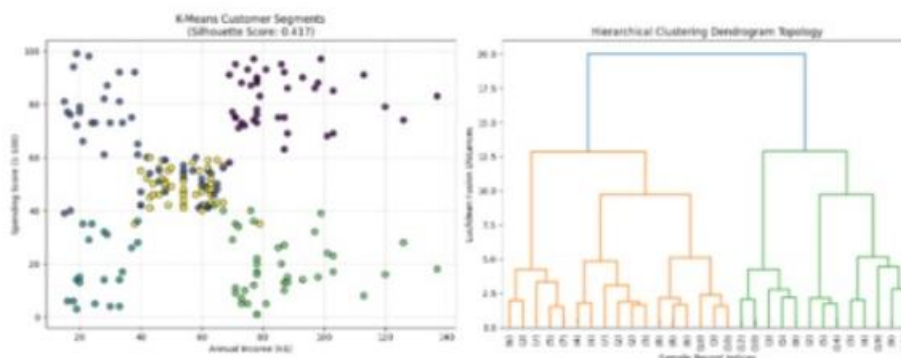
7. Generate Visual Evaluation Plots

```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 6))

# Plot 1: K-Means Spatial Clusters Scattered Map
ax1.scatter(X_features[:, 1], X_features[:, 2],
            c=kmeans_labels, cmap='viridis', edgecolor='k', s=50, alpha=0.8)
ax1.set_title(f"K-Means Customer Segments\n(Silhouette Score: {kmeans_silhouette:.3f})")
ax1.set_xlabel(df.columns[3])
ax1.set_ylabel(df.columns[4])
ax1.grid(True, linestyle='--', alpha=0.5)

# Plot 2: Hierarchical Clustering Dendrogram Topology Tree Map
ax2.set_title("Hierarchical Clustering Dendrogram Topology")
ax2.set_xlabel("Sample Record Indices")
ax2.set_ylabel("Euclidean Fusion Distances")
dendrogram(Z, truncate_mode='lastp', p=30, ax=ax2, leaf_rotation=90, leaf_font_size=10, show_c

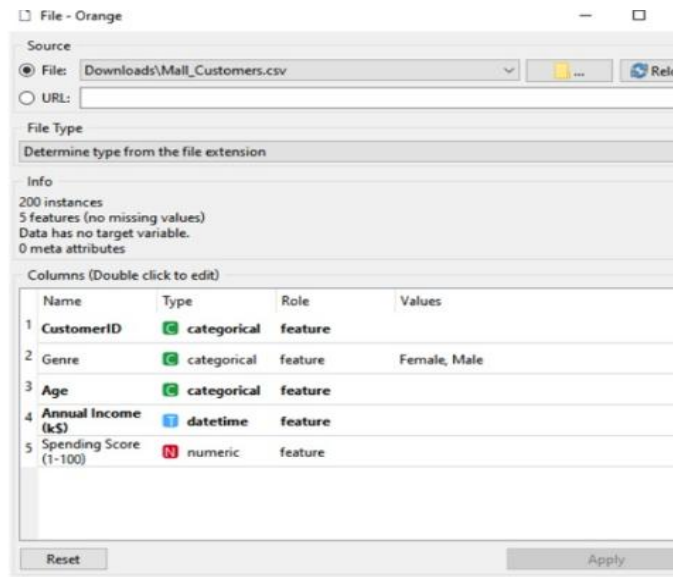
plt.tight_layout()
plt.show()
```



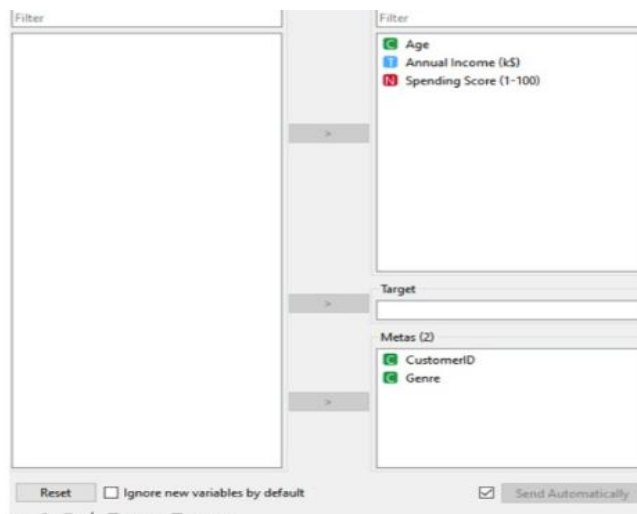
ORANGE :

WORKFLOW:

Load Data:



Preprocessing:



K MEAN Widget:

Data Table - Orange

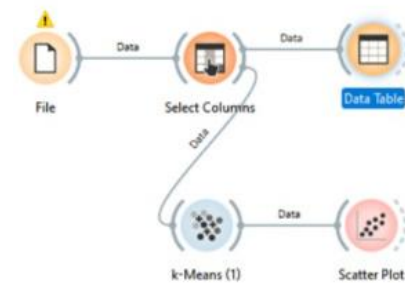
Info
200 instances (no missing data)
3 features
No target variable
2 meta attributes

Variables
☒ Show variable labels (if present)
☐ Visualize numeric values
☒ Color by instance classes

Selection
Clear Selection
☒ Select full rows

Restore Original Order
☒ Send Automatically

	CustomerID	Genre	Age	Annual Income (k\$)	Spending Score (1-100)
1	1	Male	19	15.0	39
2	2	Male	21	15.0	81
3	3	Female	20	16.0	6
4	4	Female	23	16.0	77
5	5	Female	31	17.0	40
6	6	Female	22	17.0	76
7	7	Female	35	18.0	6
8	8	Female	23	18.0	94
9	9	Male	64	19.0	3
10	10	Female	30	19.0	72
11	11	Male	67	19.0	14
12	12	Female	35	19.0	99
13	13	Female	58	20.0	15
14	14	Female	24	20.0	77
15	15	Male	37	20.0	13
16	16	Male	22	20.0	79
17	17	Female	35	21.0	35
18	18	Male	20	21.0	66
19	19	Male	52	23.0	29



CONFIGURATION AND VISUALISATION:

k-Means (1) - Orange

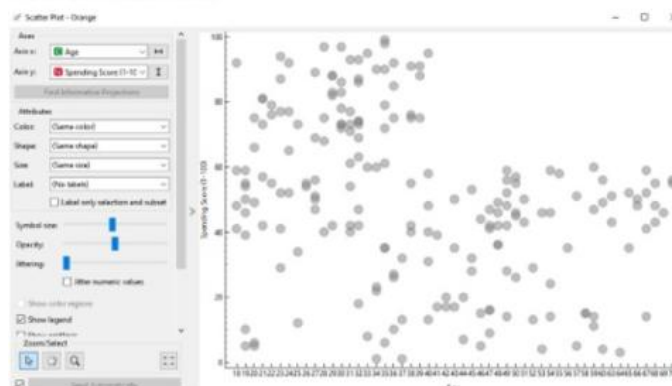
Number of Clusters
☒ Fixed:
☐ From to

Preprocessing
☒ Normalize columns

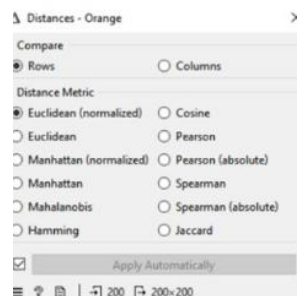
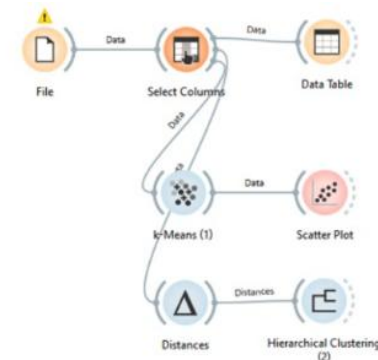
Initialization
 Initialize with KMeans++

Re-runs:
 Maximum iterations:

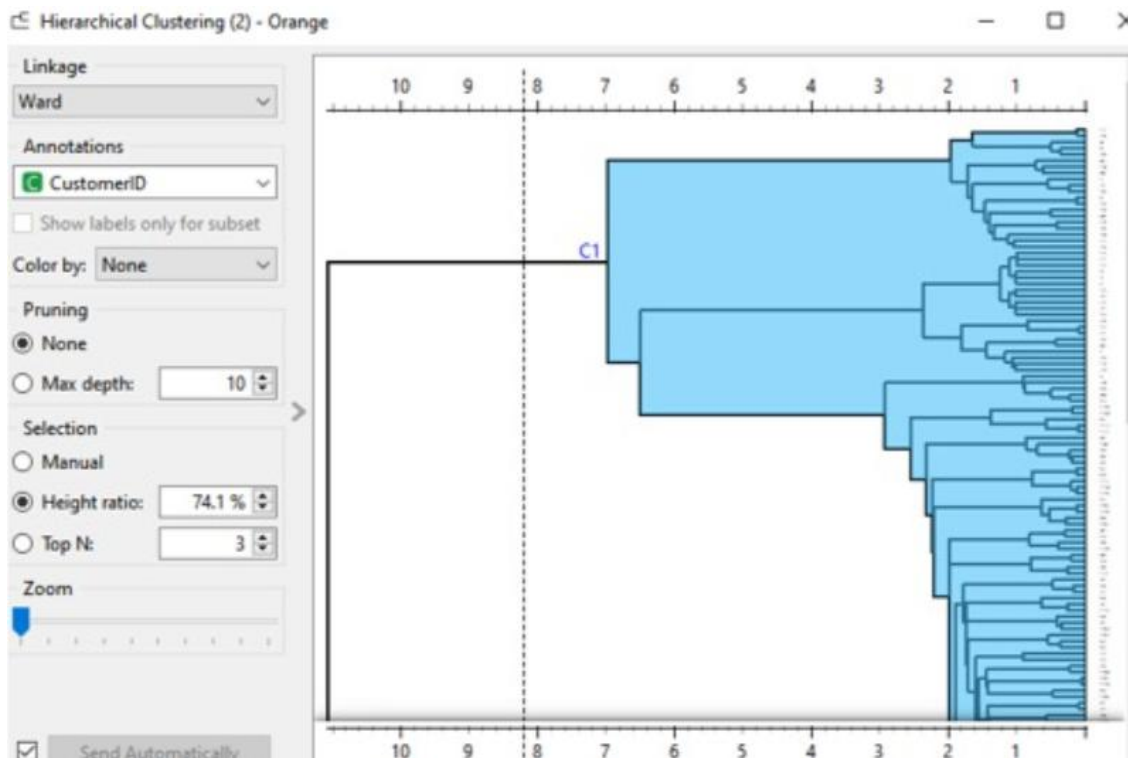
☒ Apply Automatically



Drag the Hierarchical Clustering widget.



Visualization:



Lab 13: Outlier Detection – Abnormal Banking Transactions

Objective

To detect anomalous or fraudulent transactions in a banking dataset by applying distance-based and density-based outlier detection algorithms, comparing their ability to flag unusual patterns.

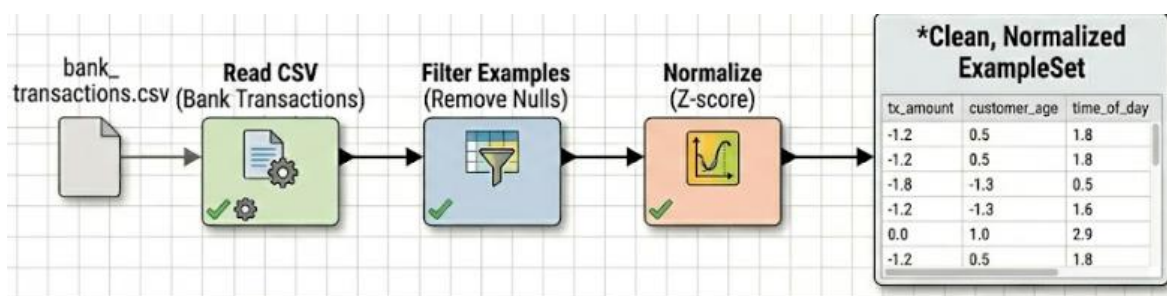
Tool & Dataset

- **Tool:** RapidMiner Studio + Python.
- **Dataset:** Banking Dataset (Kaggle).

Workflow Steps

Step 1: Data Ingestion & Preprocessing (RapidMiner)

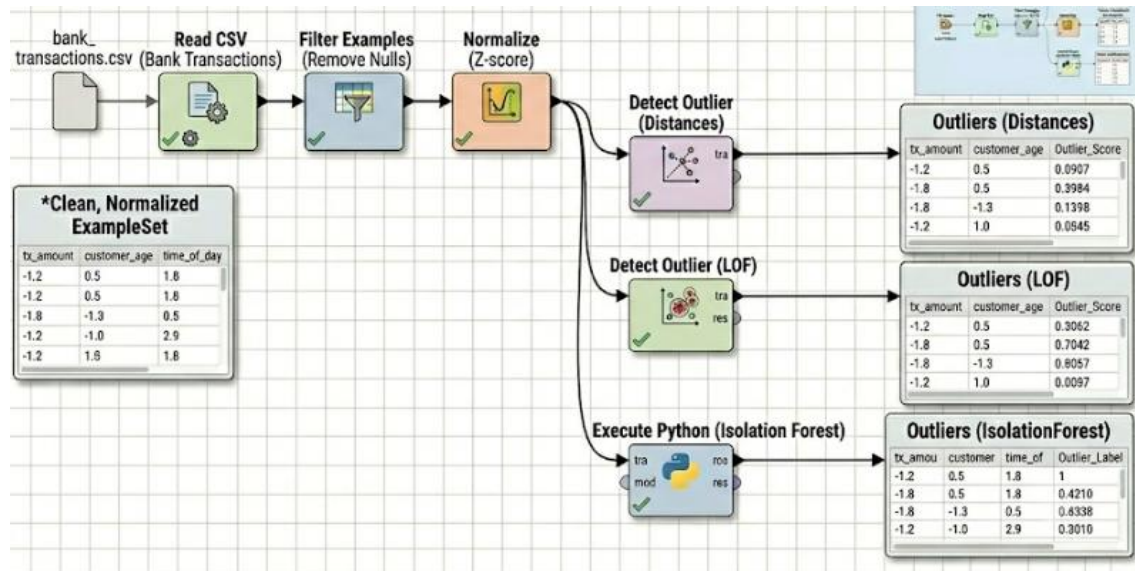
1. **Read Data:** Use the **Read CSV** operator to load the banking transaction data.
2. **Cleaning:** Use the **Filter Examples** operator to remove any null values.
3. **Scaling:** Apply the **Normalize** operator (Z-score) to ensure all features contribute equally to distance calculations, which is essential for outlier detection.



Step 2: Implementing Outlier Detection (RapidMiner)

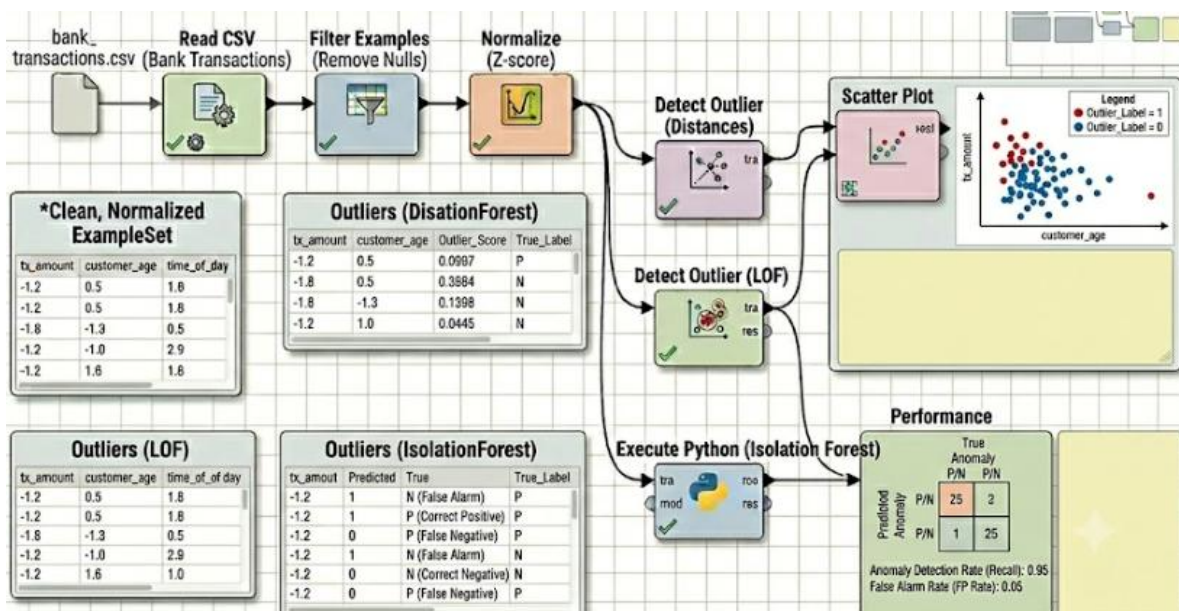
1. **Distance-Based Detection:** Use the **Detect Outlier (Distances)** operator. This identifies outliers based on the distance to their k -nearest neighbors.
2. **Density-Based Detection:** Use the **Detect Outlier (LOF)** (Local Outlier Factor) operator. This is highly effective for identifying outliers that are anomalous relative to their local neighborhood density.

3. **Isolation Forest (via Python):** Use the **Execute Python** operator to apply the Isolation Forest algorithm, which is often superior for high-dimensional anomaly detection.



Visualization & Analysis

1. **Plotting:** Use the **Scatter Plot** operator in RapidMiner, mapping the "Outlier Score" to the color of the points to visualize which transactions were flagged as anomalous.
2. **Performance:** Connect the results to a **Performance** operator to assess your "Anomaly Detection Rate" and "False Alarm Rate"



PYTHON IMPLEMENTATION:

```
import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.ensemble import IsolationForest
from sklearn.neighbors import NearestNeighbors
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import confusion_matrix
```

```
df = pd.read_csv("test.csv")
print(df.columns.tolist())
```

```
FEATURE_COL_1 = 'age'
FEATURE_COL_2 = 'balance'
```

```
GROUND_TRUTH_COL = 'y'
```

```
['age';'job';'marital';'education';'default';'balance';'housing';'loan';'contact';'day';'month';'duration';'campaign';'pdays';'previous';'poutcome';'y']
```

1. DATA LOADING & PREPROCESSING

```
if not os.path.exists(FILE_PATH):
    raise FileNotFoundError(f"Could not find the dataset file at: {FILE_PATH}.")

# CRITICAL FIX: Added sep=';' to correctly split your columns
df = pd.read_csv(FILE_PATH, sep=';')

# Clean up quotes from string columns if they exist natively in the data entries
for col in df.select_dtypes(include=['object']).columns:
    df[col] = df[col].astype(str).str.replace('"', '').str.replace("'", '')

print(f"Dataset loaded successfully! Total Rows: {df.shape[0]}, Total Columns: {df.shape[1]}")
print("Parsed Columns:", df.columns.tolist())

# Drop missing values safely
df = df.dropna(subset=[FEATURE_COL_1, FEATURE_COL_2, GROUND_TRUTH_COL]).reset_index(drop=True)

# Convert Ground Truth 'yes'/'no' into 1 and 0 so the KPI functions can read it
df['Encoded_Ground_Truth'] = df[GROUND_TRUTH_COL].map({'yes': 1, 'no': 0})

# Extract features and scale them
X = df[[FEATURE_COL_1, FEATURE_COL_2]]
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

```
Dataset loaded successfully! Total Rows: 4521, Total Columns: 17
Parsed Columns: ['age', 'job', 'marital', 'education', 'default', 'balance', 'housing', 'loan', 'contact', 'day', 'month', 'duration', 'campaign', 'pdays', 'previous', 'poutcome', 'y']
```


2. ALGORITHM 1: ISOLATION FOREST

```
iso_forest = IsolationForest(contamination=0.05, random_state=42)
df['IF_Raw_Pred'] = iso_forest.fit_predict(X_scaled)
df['IF_Anomaly'] = df['IF_Raw_Pred'].map([1: 0, -1: 1])
```

3. ALGORITHM 2: DISTANCE-BASED (KNN DISTANCE)

```
k_neighbors = 5
knn = NearestNeighbors(n_neighbors=k_neighbors)
knn.fit(X_scaled)
distances, _ = knn.kneighbors(X_scaled)

df['KNN_Outlier_Score'] = distances.mean(axis=1)
contamination_threshold = np.percentile(df['KNN_Outlier_Score'], 95)
df['KNN_Anomaly'] = (df['KNN_Outlier_Score'] >= contamination_threshold).astype(int)
```

4. KPI & EVALUATION METRICS CALCULATION

```
def calculate_lab_kpis(y_true, y_pred):
    tn, fp, fn, tp = confusion_matrix(y_true, y_pred).ravel()
    anomaly_detection_rate = (tp / (tp + fn)) * 100 if (tp + fn) > 0 else 0
    false_alarm_rate = (fp / (fp + tn)) * 100 if (fp + tn) > 0 else 0
    return anomaly_detection_rate, false_alarm_rate

# Compute metrics using our encoded 1/0 column
if_adr, if_far = calculate_lab_kpis(df['Encoded_Ground_Truth'], df['IF_Anomaly'])
knn_adr, knn_far = calculate_lab_kpis(df['Encoded_Ground_Truth'], df['KNN_Anomaly'])

kpi_table = pd.DataFrame({
    "Evaluation Metric (KPI)": ["Anomaly Detection Rate (ADR)", "False Alarm Rate (FAR)"],
    "Isolation Forest": [f"{if_adr:.2f}%", f"{if_far:.2f}%"],
    "Distance-Based (KNN)": [f"{knn_adr:.2f}%", f"{knn_far:.2f}%"]
})

print(" EVALUATION METRICS")
print(kpi_table.to_string(index=False))
```

```
EVALUATION METRICS
Evaluation Metric (KPI) Isolation Forest Distance-Based (KNN)
Anomaly Detection Rate (ADR) 8.06% 7.40%
False Alarm Rate (FAR) 4.60% 4.70%
```

Lab 14: Scraping & Sentiment Analysis – Product Review Analysis

Objective

To extract (scrape) product reviews from social media or review platforms, clean the unstructured text, and perform sentiment classification to determine polarity (positive, negative, neutral).

Tool & Dataset

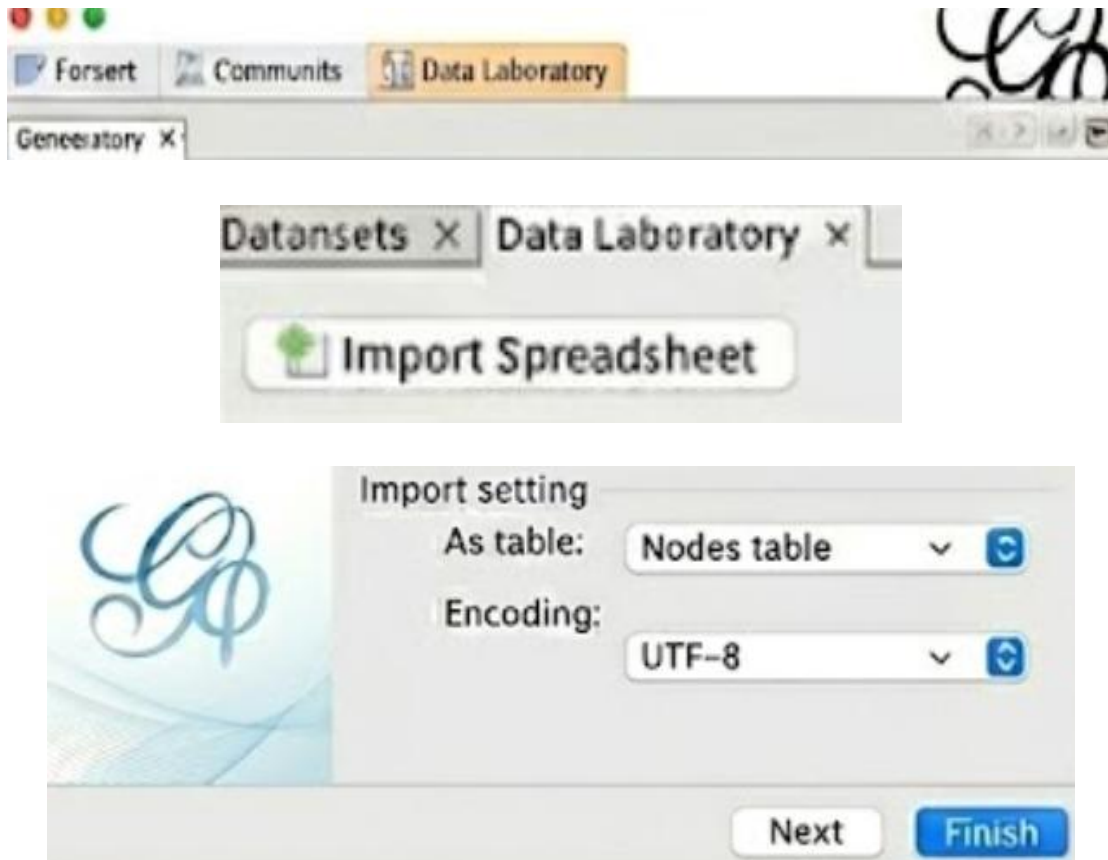
- **Tool:** CiteSpace / Gephi + Python.
- **Dataset:** Social Media / Review Dataset.

Workflow Steps

Step 1 – Data Import

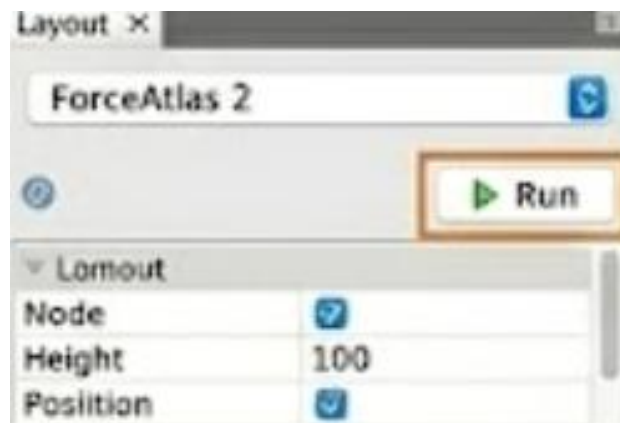
- Import the dataset (e.g., scraped reviews) into Gephi using **Import Spreadsheet**.
- Data is read as a **Nodes Table** (entities such as words or users) and **Edges Table** (connections between them).
- Encoding is set to **UTF-8** to ensure proper handling of text data.

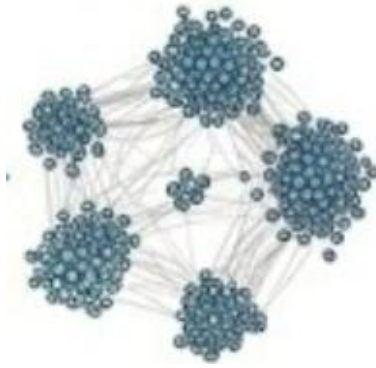




Step 2 – Network Layout

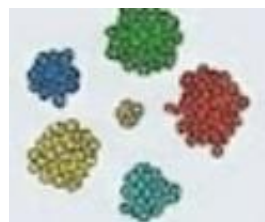
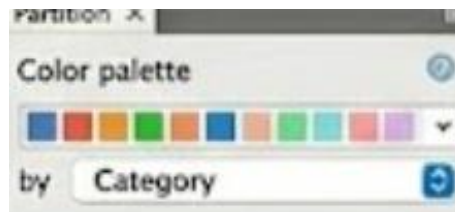
- Apply the **ForceAtlas 2** layout algorithm to arrange nodes visually.
- This algorithm simulates physical forces: connected nodes attract each other, while unrelated nodes repel.
- The result is a clear cluster structure showing groups of related words or reviewers.





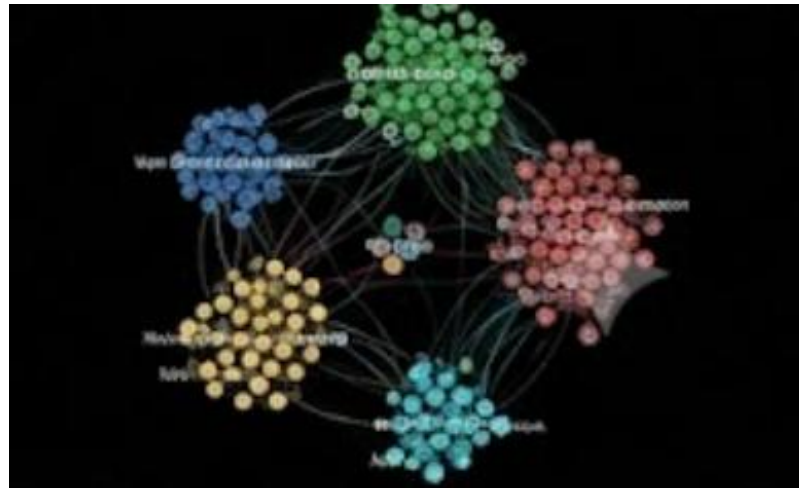
Step 3 – Color & Categorization

- Use the **Color Palette** option to assign colors by category (e.g., sentiment polarity or topic).
- Each color represents a distinct sentiment cluster — for example, blue for positive, red for negative, and yellow for neutral.



Step 4 – Visualization & Export

- After running the layout, visualize the network graph.
- Export the final visualization using the **Export** window (filename, format, etc.).
- The output shows interconnected clusters representing how sentiments or topics relate across reviews.



PYTHON IMPLEMENTATION :

1. Load and Clean Data

```
import pandas as pd
import re

# Load data with encoding flx
df = pd.read_csv("test (1).csv", encoding='latin-1')
text_col = 'text' # Update if your column name is different

# Simple cleaning function
def clean(text):
    text = re.sub(r'[^a-zA-Z\s]', '', str(text).lower())
    return [w for w in text.split() if w not in ['the', 'and', 'a', 'to', 'of']]

df['clean_tokens'] = df[text_col].apply(clean)
```

2. Sentiment Scoring Engine

```
pos = ['great', 'love', 'amazing', 'best', 'nice', 'happy']
neg = ['worst', 'hate', 'terrible', 'poor', 'slow', 'error']

def get_polarity(tokens):
    p = 0
    n = 0
    for w in tokens:
        if w in pos:
            p += 1
        elif w in neg:
            n += 1
    return (p - n) / (p + n) if (p + n) > 0 else 0

df['clean_tokens'].apply(get_polarity)
df['score'] = df['clean_tokens'].apply(lambda s: 'Positive' if s > 0 else ('Negative' if s < 0 else 'Neutral'))
```

3. KPI Metrics Calculation ¶

```
# KPI 1: Distribution
dist = df['sentiment'].value_counts(normalize=True) * 100
print("--- Sentiment Distribution (%) ---")
print(dist)

# KPI 2: Polarity Summary
print("\n--- Polarity Statistics ---")
print(f"Avg Polarity: {df['score'].mean():.4f}")
print(f"Max Polarity: {df['score'].max():.4f}")
print(f"Min Polarity: {df['score'].min():.4f}")
```

```
--- Sentiment Distribution (%) ---
sentiment
Neutral      85.503634
Positive     12.107996
Negative       2.388370
Name: proportion, dtype: float64

--- Polarity Statistics ---
Avg Polarity: 0.0973
Max Polarity: 1.0000
Min Polarity: -1.0000
```

4. Visualization

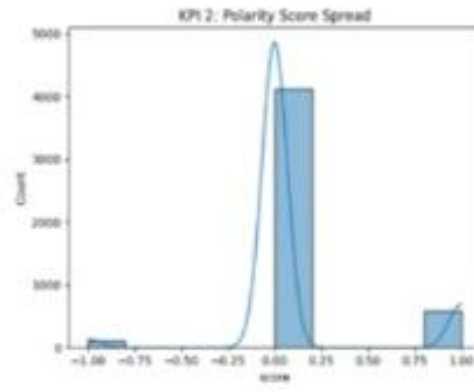
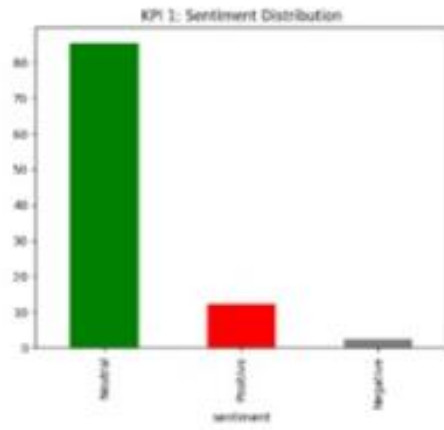
```
import matplotlib.pyplot as plt
import seaborn as sns

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

# Plot Distribution
dist.plot(kind='bar', ax=ax1, color=['green', 'red', 'gray'])
ax1.set_title("KPI 1: Sentiment Distribution")

# Plot Polarity Spread
sns.histplot(df['score'], ax=ax2, bins=10, kde=True)
ax2.set_title("KPI 2: Polarity Score Spread")

plt.show()
```

Lab 15: Final Capstone – End-to-End Business Solution

Objective

To design, implement, and document a complete data science pipeline—from data acquisition and preprocessing to modeling, comparison, and business recommendation—using a real-world dataset of your choice.

1. Data Preprocessing

Before training any machine-learning model, raw data must be cleaned and transformed into a usable format.

- **Raw Features:** These are the original attributes collected from datasets (e.g., numeric, categorical, or text values). They often contain inconsistencies, missing values, or varying scales.
- **Processed Data:** After cleaning, the dataset becomes structured and ready for analysis. Each record is standardized, and irrelevant or noisy features are removed.
- **Feature Scaling:** Ensures that all attributes contribute equally to the model's learning process. Without scaling, features with larger numeric ranges can dominate the training process.

Before

	LMM_values	skint_values	obdlo_sabes	oodly_sabes
ramov 1	1600	1600	1400	1600
ramov 2	1600	1600	1400	1600
ramov 3	1500	1000	1500	1600
ramov 4	1500	1770	1500	1600
ramov 5	1600	1600	1500	1600

After

After	Status
5 600	Piccessed
5 600	Piccessed
5 600	Piccessed
5 600	Piccessed
5 600	Piccessed

This block contains two screenshots from a data science application. The left screenshot shows a table titled 'MODELS' with columns 'Models' and 'Status'. It lists ten models with their corresponding status values. The right screenshot shows two panels: '8 RAW FEATURES' and '9 SCALED FEATURES'. The 'RAW FEATURES' panel includes a trash can icon and three charts. The 'SCALED FEATURES' panel includes a table with columns 'Label', 'Rogrlit', 'Seamis', and 'Scaled', showing values for various labels.

MODELS	
Models	Status
1 DECISION TREE	0.35
2 NAIVE BAYES	0.34
3 MKN	0.35
4 MYN	8.33
5 SVM	0.99
6 LOGISTIC BEGRESSION	0.95
7 RANDOM FOREST	0.82
8 GRADIENT FOREST	0.36
9 GRADIENT BOOSTING	0.32
10 NEURAL NETWORK	0.92

8 RAW FEATURES			
	Rogrlit	Seamis	Scaled
Label 3	0.3587	-0.000	6.000
Label 4	9.5587	0.000	0.000
Label d	0.4950	0.000	0.000
Utike,Dolas	0.4200	4.000	0.000
Label 3	1.1988	3.900	0.300
Label 11	0.3991	2.000	0.078

2. Z- Score Normalization

Z- Score normalization (also called standardization) transforms data so that each feature has a mean = 0 and standard deviation = 1. The formula is:

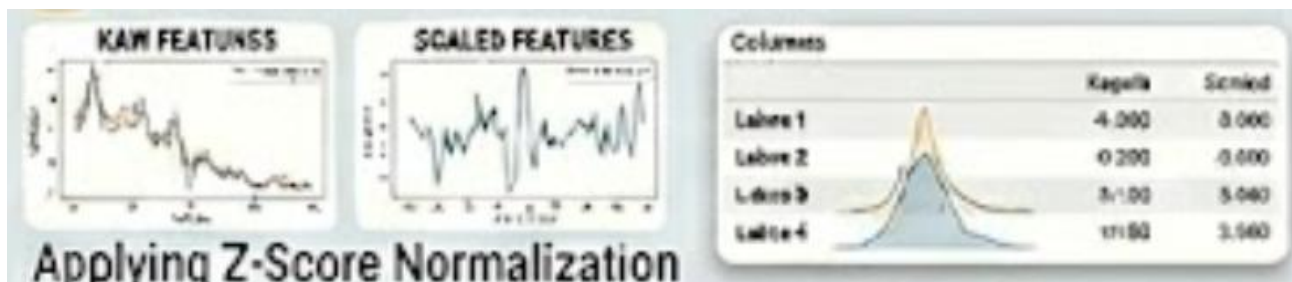
$$Z = \frac{X - \mu}{\sigma}$$

where X is the original value, μ is the mean, and σ is the standard deviation.

Purpose:

- Removes bias caused by differing scales.
- Improves convergence speed for algorithms like SVM, KNN, and Neural Networks.
- Makes data comparable across features.

Effect: After applying Z- Score normalization, the dataset changes from “Before” (raw numeric values) to “After” (scaled values). Graphs of raw vs. scaled features show smoother distributions centered around 0.



3. Model Training

Once data is normalized, multiple algorithms are trained to find the best predictive model. Common models include:

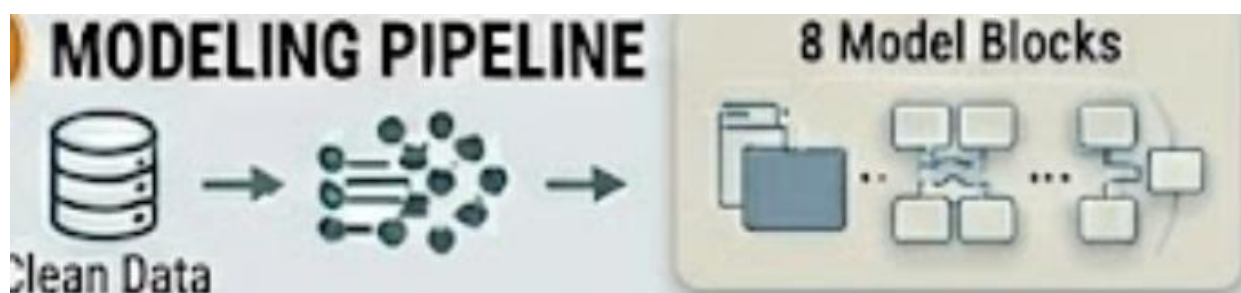
- **Decision Tree:** Splits data based on attribute values to form hierarchical decision rules.
- **Naive Bayes:** Probabilistic classifier based on Bayes' theorem; assumes feature independence.
- **K- Nearest Neighbor (KNN):** Classifies data based on proximity to neighboring points.
- **Support Vector Machine (SVM):** Finds the optimal hyperplane separating classes.

- **Logistic Regression:** Models binary outcomes using a logistic function.
- **Random Forest:** Ensemble of decision trees that improves accuracy and reduces overfitting.
- **Gradient Boosting:** Sequentially builds models to correct previous errors.
- **Neural Network:** Mimics human brain structure using interconnected layers for complex pattern learning.



4. Modeling Pipeline

The **modeling pipeline** automates the workflow: **Clean Data** → **Feature Scaling** → **Model Training** → **Evaluation** → **Comparison**. Each model block receives the same processed dataset, ensuring fair comparison.



5. Model Evaluation

Performance is measured using key metrics:

Metric **Description**

Accuracy Percentage of correctly predicted instances.

Precision Ratio of true positives to all predicted positives.

Recall Ratio of true positives to all actual positives.

F1-Score Harmonic mean of precision and recall.

ROC-AUC Measures the model's ability to distinguish between classes.

The **Model Comparison Table** ranks algorithms based on these metrics.

- Models like **Naive Bayes**, **KNN**, and **Gradient Boosting** often achieve high ROC-AUC scores.
- The **Best Model** is selected based on balanced performance across all metrics.

MODEL COMPARISON TABLE					
	Accuracy	Precision	Recall	F1-Score	ROC-AUC
(1) Decision Tree	0.99	0.92	0.99	0.98	0.99
(2) Naive Bayes	0.87	0.86	0.87	0.76	0.93
(2) KNN	0.96	0.83	0.83	0.56	0.96
(4) GYM	0.53	0.55	0.52	0.76	6.49
(5) Logistic Regression	0.95	0.95	0.95	0.58	0.98
(6) Random Forest	0.04	0.56	0.83	0.57	0.96
(7) Gradient Boosting	0.93	0.86	0.55	0.56	0.37
(8) Neural Network	0.83	0.53	0.34	0.55	0.39

6. Outcome

After evaluation, the workflow identifies the most accurate and reliable model for prediction. The **ROC-AUC Comparison** and **Best Model Graph** visualize

performance differences, helping analysts choose the optimal algorithm for deployment.



IMPLEMENTATION ON PYTHON:

UPLOAD DATASET

```
[1]
✓ 3m
from google.colab import files

uploaded = files.upload()

Choose Files Churn_Modelling.csv
Churn_Modelling.csv(text/csv) - 684858 bytes, last modified: 6/16/2026 - 100% done
Saving Churn_Modelling.csv to Churn_Modelling.csv
```

IMPORT LIBRARIES

```
[2]
✓ 2s
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import accuracy_score
```


LOAD DATASET

[3]
✓ Os

```
df = pd.read_csv('Churn_Modelling.csv')  
  
print(df.head())  
print(df.shape)
```

▼

	RowNumber	CustomerId	Surname	CreditScore	Geography	Gender	Age	\
0	1	15634602	Hargrave	619	France	Female	42	
1	2	15647311	Hill	608	Spain	Female	41	
2	3	15619304	Onio	502	France	Female	42	
3	4	15701354	Boni	699	France	Female	39	
4	5	15737888	Mitchell	850	Spain	Female	43	

	Tenure	Balance	NumOfProducts	HasCrCard	IsActiveMember	\
0	2	0.00	1	1	1	
1	1	83807.86	1	0	1	
2	8	159660.80	3	1	0	
3	1	0.00	2	0	0	
4	2	125510.82	1	1	1	

	EstimatedSalary	Exited
0	101348.88	1
1	112542.58	0
2	113931.57	1
3	93826.63	0
4	79084.10	0

(10000, 14)

DATA PREPROCESSING STEPS

[4]
✓ Os

```
# Remove unnecessary columns  
df = df.drop(['RowNumber', 'CustomerId', 'Surname'], axis=1)  
  
# Encode categorical variables  
le = LabelEncoder()  
  
df['Geography'] = le.fit_transform(df['Geography'])  
df['Gender'] = le.fit_transform(df['Gender'])  
  
# Features and Target  
X = df.drop('Exited', axis=1)  
y = df['Exited']  
  
print(X.head())
```

▼

	CreditScore	Geography	Gender	Age	Tenure	Balance	NumOfProducts	\
0	619	0	0	42	2	0.00	1	
1	608	2	0	41	1	83807.86	1	
2	502	0	0	42	8	159660.80	3	
3	699	0	0	39	1	0.00	2	
4	850	2	0	43	2	125510.82	1	

	HasCrCard	IsActiveMember	EstimatedSalary
0	1	1	101348.88
1	0	1	112542.58
2	1	0	113931.57
3	0	0	93826.63
4	1	1	79084.10

TRAIN-TEST SPLIT

```
[5]  
✓ 0s  
X_train, X_test, y_train, y_test = train_test_split(  
    X,  
    y,  
    test_size=0.2,  
    random_state=42  
)
```

IMPORT MODELS

```
[6]  
✓ 1s  
from sklearn.linear_model import LogisticRegression  
from sklearn.tree import DecisionTreeClassifier  
from sklearn.ensemble import RandomForestClassifier  
from sklearn.neighbors import KNeighborsClassifier  
from sklearn.naive_bayes import GaussianNB  
from sklearn.svm import SVC  
from sklearn.neural_network import MLPClassifier  
from sklearn.ensemble import GradientBoostingClassifier
```

TRAIN 8 MODELS

```
7]  
' 13s  
models = {  
    "Logistic Regression": LogisticRegression(max_iter=1000),  
    "Decision Tree": DecisionTreeClassifier(),  
    "Random Forest": RandomForestClassifier(),  
    "KNN": KNeighborsClassifier(),  
    "Naive Bayes": GaussianNB(),  
    "SVM": SVC(),  
    "Neural Network": MLPClassifier(max_iter=500),  
    "Gradient Boosting": GradientBoostingClassifier()  
}  
  
results = []  
  
for name, model in models.items():  
    model.fit(X_train, y_train)  
  
    predictions = model.predict(X_test)  
  
    accuracy = accuracy_score(y_test, predictions)  
  
    results.append([name, accuracy])  
  
print(name, "Accuracy =", round(accuracy*100,2), "%")
```

```

n_iter_i = _check_optimize_result(
Logistic Regression Accuracy = 81.6 %
Decision Tree Accuracy = 78.5 %
Random Forest Accuracy = 86.5 %
KNN Accuracy = 76.4 %
Naive Bayes Accuracy = 79.0 %
SVM Accuracy = 80.35 %
Neural Network Accuracy = 45.0 %
Gradient Boosting Accuracy = 86.55 %

```

COMPARISON TABLE

```

[8]
✓ Os
results_df = pd.DataFrame(
    results,
    columns=["Model", "Accuracy"]
)

results_df = results_df.sort_values(
    by="Accuracy",
    ascending=False
)

print(results_df)

```

```

...
      Model  Accuracy
7  Gradient Boosting  0.8655
2    Random Forest  0.8650
0  Logistic Regression  0.8160
5              SVM  0.8035
4    Naive Bayes  0.7900
1    Decision Tree  0.7850
3              KNN  0.7640
6    Neural Network  0.4500

```

BEST MODEL SELECTION

```

[9]
✓ Os
best_model = results_df.iloc[0]

print("Best Model:", best_model['Model'])
print("Best Accuracy:", round(best_model['Accuracy']*100,2), "%")

```

✓ Best Model: Gradient Boosting
 Best Accuracy: 86.55 %

ACCURACY COMPARISON GRAPH

[10]
✓ 0s

```
plt.figure(figsize=(10,5))

plt.bar(
    results_df["Model"],
    results_df["Accuracy"]
)

plt.xticks(rotation=45)

plt.ylabel("Accuracy")

plt.title("Model Comparison")

plt.show()
```

